

Strategic Copilot

Ouissame Mahtour

2026

Table des matières

Résumé	9
Abstract	10
Introduction Générale	12
1 Contexte général du projet et analyse des besoins	16
1.1 Introduction	16
1.2 Présentation de l'entreprise d'accueil	17
1.3 Contexte du stage	18
1.4 Problématique	18
1.5 Objectifs du projet	20
1.6 Présentation générale de Strategic Copilot	21
1.6.1 Vision et positionnement	21
1.6.2 Les trois espaces utilisateurs	21
1.6.3 Fonctionnalités transverses	23
1.7 Analyse des besoins	23
1.7.1 Acteurs et cas d'usage	23
1.7.2 Besoins fonctionnels	24
1.7.3 Besoins non fonctionnels	25
1.7.4 Contraintes identifiées	26
1.8 Conclusion	27
2 Étude Bibliographique	28

2.1	Introduction	28
2.2	Transformation digitale des ressources humaines	29
2.2.1	Enjeux de la digitalisation RH	29
2.2.2	Systèmes d'information RH modernes	30
2.3	Gestion des talents et des compétences	31
2.3.1	Talent Management	31
2.3.2	Skill Management	31
2.3.3	Workforce Planning	32
2.4	Gestion de portefeuille de projets	33
2.4.1	Project Portfolio Management	33
2.4.2	Allocation des ressources	33
2.4.3	Indicateurs de performance et de viabilité	34
2.5	Intelligence artificielle appliquée à l'aide à la décision	35
2.5.1	Systèmes d'aide à la décision	35
2.5.2	Large Language Models	35
2.5.3	Recommandations automatisées	36
2.5.4	Architectures multi-agents et Agentic AI	37
2.6	Automatisation des processus métier	37
2.6.1	Workflow Automation	37
2.6.2	Orchestration avec n8n	38
2.7	Technologies web modernes	38
2.7.1	React.js et Vite	38
2.7.2	Tailwind CSS	39
2.7.3	Supabase et PostgreSQL	40
2.7.4	OpenAI et IA générative	40
2.7.5	Visualisation de données avec Recharts	41
2.7.6	Sécurité, authentification et déploiement SaaS	41
2.8	Synthèse et positionnement de Strategic Copilot	42
2.9	Conclusion	43

3	Cahier des charges et planification	44
3.1	Introduction	44
3.2	Cahier des charges	44
3.2.1	Présentation générale du système	44
3.2.2	Acteurs du système	45
3.2.3	Fonctionnalités principales	45
3.2.4	Contraintes techniques et organisationnelles	46
3.3	Spécifications fonctionnelles	47
3.3.1	Fonctionnalités de l'espace Manager	47
3.3.2	Fonctionnalités de l'espace RH	48
3.3.3	Fonctionnalités de l'espace Talent	48
3.3.4	Fonctionnalités du Copilot intelligent	49
3.4	Spécifications non fonctionnelles	49
3.4.1	Performance	49
3.4.2	Sécurité	50
3.4.3	Fiabilité	50
3.4.4	Maintenabilité	50
3.4.5	Scalabilité	50
3.4.6	Ergonomie et expérience utilisateur	50
3.5	Planification du projet	51
3.5.1	Méthodologie adoptée	51
3.5.2	Découpage en phases	51
3.5.3	Diagramme de Gantt	52
3.5.4	Livrables du projet	52
3.6	Gestion des risques	53
3.6.1	Risques techniques	53
3.6.2	Risques organisationnels	53
3.6.3	Mesures d'atténuation	53
3.7	Ressources utilisées	53

3.7.1	Ressources humaines	53
3.7.2	Ressources logicielles	54
3.7.3	Ressources matérielles	54
3.8	Conclusion	54
4	Conception	55
4.1	Introduction	55
4.2	Architecture globale du système	56
4.2.1	Vue d'ensemble	56
4.2.2	Architecture multi-couches	56
4.2.3	Architecture multi-agents	57
4.3	Conception fonctionnelle	57
4.3.1	Acteurs du système	57
4.3.2	Cas d'usage	58
4.3.3	Flux fonctionnels	58
4.4	Conception de l'architecture agentique	59
4.4.1	Agents d'analyse	59
4.4.2	Agents de recommandation	59
4.4.3	Agent orchestrateur	59
4.4.4	Interaction entre agents	60
4.5	Conception des workflows n8n	60
4.5.1	Principes d'orchestration	60
4.5.2	Description détaillée des workflows réellement présents	61
4.5.3	Communication entre workflows	61
4.5.4	Gestion des erreurs et reprise	61
4.6	Conception de la base de données	63
4.6.1	Architecture Supabase/PostgreSQL	63
4.6.2	Tables principales	63
4.6.3	Relations entre entités	63
4.7	Conception du frontend	64

4.7.1	Architecture React/Vite	64
4.7.2	Organisation des composants	64
4.7.3	Gestion des états	64
4.7.4	Workspaces Manager, RH et Talent	65
4.8	Conception des APIs	65
4.8.1	Endpoints exposés	65
4.8.2	Structure des réponses JSON	66
4.8.3	Intégration avec React Query	66
4.9	Sécurité et gouvernance	67
4.9.1	Authentification	67
4.9.2	Contrôle d'accès	67
4.9.3	Traçabilité des décisions IA	67
4.10	Conclusion	68
5	Réalisation et implémentation	69
5.1	Introduction	69
5.2	Environnement de développement	69
5.2.1	Outils utilisés	69
5.2.2	Technologies retenues	70
5.2.3	Configuration de l'environnement	70
5.3	Implémentation du frontend	71
5.3.1	Structure du projet React/Vite	71
5.3.2	Gestion du routage	72
5.3.3	Gestion des états avec React Query	72
5.3.4	Design system et Tailwind CSS	73
5.3.5	Workspaces Manager, RH et Talent	73
5.4	Implémentation du backend	73
5.4.1	Architecture générale des workflows n8n	73
5.4.2	Principaux workflows réellement implémentés	74
5.4.3	Intégration des services OpenAI	74

5.4.4	Exposition des APIs via Webhooks	75
5.5	Implémentation de la base de données	75
5.5.1	Structure Supabase/PostgreSQL	75
5.5.2	Principales tables	75
5.5.3	Contraintes et intégrité des données	76
5.6	Interfaces principales	76
5.6.1	Tableau de bord Manager	76
5.6.2	Espace RH	76
5.6.3	Espace Talent	77
5.6.4	Mission Control	77
5.6.5	Copilot conversationnel	77
5.6.6	Simulateur What-if	78
5.7	Fonctionnalités avancées	78
5.7.1	Alertes intelligentes	78
5.7.2	Recommandations IA	78
5.7.3	Journal des décisions	78
5.7.4	Génération de rapports	79
5.8	Difficultés rencontrées et solutions apportées	79
5.9	Conclusion	80
6	Tests et résultats	81
6.1	Introduction	81
6.2	Stratégie de test	81
6.2.1	Objectifs des tests	81
6.2.2	Méthodologie adoptée	82
6.2.3	Environnement de test	82
6.3	Tests fonctionnels	83
6.3.1	Authentification et gestion des rôles	83
6.3.2	Tests de l'espace Manager	83
6.3.3	Tests de l'espace RH	84

6.3.4	Tests de l'espace Talent	84
6.3.5	Tests du Copilot intelligent	84
6.4	Tests des APIs et des workflows n8n	85
6.4.1	Validation des endpoints Webhook	85
6.4.2	Tests via Postman	85
6.4.3	Validation des workflows multi-agents	86
6.5	Tests de la base de données	86
6.5.1	Intégrité des données	86
6.5.2	Contraintes relationnelles	87
6.5.3	Cohérence des calculs	87
6.6	Cas d'usage complet	87
6.6.1	Analyse d'un projet réel	87
6.6.2	Calcul du score de viabilité	87
6.6.3	Recommandation Continue / Adjust / Stop	88
6.6.4	Simulation What-if	88
6.7	Résultats obtenus	88
6.7.1	Résultats fonctionnels	88
6.7.2	Résultats techniques	89
6.7.3	Apports métier	89
6.8	Limites observées	89
6.9	Perspectives d'amélioration	89
6.10	Conclusion	90
	Conclusion générale	91
	Bibliographie	94
	Annexes	96

Remerciements

Avant toute chose, je rends grâce à Dieu Tout-Puissant pour m'avoir accordé la santé, la force, la patience et la persévérance nécessaires pour mener à bien ce projet de fin d'études.

Je tiens à exprimer ma profonde gratitude à Madame Soundouss Abroun, mon encadrante pédagogique, pour la qualité de son accompagnement, sa disponibilité constante, ses conseils judicieux et la rigueur scientifique qu'elle m'a transmise tout au long de ce travail.

J'adresse également mes plus sincères remerciements à Monsieur Youssef Moufak, ainsi qu'à Monsieur Hamza Sadouk, Monsieur Hicham Taoufik et Monsieur Mohyi Eddine Saadani, mes encadrants en entreprise, pour leur accueil chaleureux, leur confiance, leur disponibilité et les précieux conseils qu'ils m'ont prodigués durant cette expérience. Leurs orientations pertinentes et leur expertise ont grandement contribué à la réussite de ce projet.

Mes remerciements s'adressent aussi à l'ensemble des enseignants de l'École Marocaine des Sciences de l'Ingénieur de Tanger, qui ont assuré ma formation avec dévouement et professionnalisme, et qui m'ont permis d'acquérir les connaissances techniques et méthodologiques indispensables à mon parcours.

Je tiens à exprimer une reconnaissance toute particulière à ma famille pour son soutien inconditionnel, sa patience, ses encouragements constants et la confiance qu'elle m'a toujours témoignée. Sa présence a été une source de motivation essentielle tout au long de mes études.

Enfin, j'adresse mes remerciements à toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce projet et ont participé à l'enrichissement de cette expérience académique et humaine.

À toutes et à tous, j'exprime ma sincère gratitude et ma profonde reconnaissance.

Résumé

Dans un environnement organisationnel marqué par la diversification des compétences, l'évolution rapide des projets et la nécessité d'un pilotage transversal, les entreprises ont besoin d'outils capables de centraliser l'information et d'anticiper les déséquilibres. Le présent travail s'inscrit dans ce contexte et porte sur Strategic Copilot : plateforme SaaS intelligente d'aide à la décision pour la gestion stratégique des talents et des projets. La plateforme accompagne les managers, les responsables RH et les talents au moyen de tableaux de bord interactifs, d'alertes intelligentes et de recommandations issues de l'intelligence artificielle, afin de piloter efficacement les compétences, les charges de travail et le portefeuille de projets.

L'objectif de ce projet de fin d'études est de concevoir et de réaliser une solution web intégrée offrant la visualisation des indicateurs RH et projet, la détection des risques (surcharge, fin de contrat, manque de compétences), l'analyse de la viabilité des projets, la génération de recommandations automatisées par IA et la gestion des demandes RH au sein de workflows décisionnels automatisés, avec une expérience adaptée à chaque profil utilisateur.

La démarche adoptée combine l'analyse des besoins, la définition des cas d'usage par espace métier (manager, RH, talent), la conception de l'architecture applicative, puis le développement itératif d'une interface React.js avec Vite et Tailwind CSS. L'intégration repose sur Supabase et PostgreSQL pour la persistance, l'authentification et les API, sur n8n pour l'orchestration des flux, et sur le modèle Llama pour l'assistance décisionnelle. Des phases de tests et de validation ont permis d'ajuster les fonctionnalités sur les scénarios prioritaires.

Les travaux ont abouti à une plateforme opérationnelle structurée en espaces dédiés : pilotage des projets et des risques pour les managers, gestion des talents, des écarts de compétences et des demandes pour les RH, suivi des missions, tâches et formations pour les talents. Les tableaux de bord, les alertes et le copilot intégré confirment la pertinence de l'aide à la décision ; les workflows automatisés renforcent la traçabilité des arbitrages. L'ensemble démontre la faisabilité d'une solution SaaS au service du pilotage stratégique des ressources humaines et des projets.

Mots-clés : SaaS, Intelligence artificielle, Gestion des talents, Gestion de projets, Aide à la décision.

Abstract

In an organizational context shaped by diverse skills, fast-changing projects and the need for cross-functional oversight, companies require tools that consolidate information and anticipate imbalances. This work focuses on Strategic Copilot : an intelligent SaaS decision-support platform for strategic talent and project management. The platform supports managers, HR staff and talents through interactive dashboards, smart alerts and AI-based recommendations to manage skills, workloads and the project portfolio effectively.

The objective of this end-of-study project is to design and implement an integrated web solution that provides HR and project KPI visualization, risk detection (overload, contract expiry, skill gaps), project viability analysis, automated AI recommendations and HR request management within automated decision workflows, with an experience tailored to each user profile.

The approach combines requirements analysis, use-case definition per business workspace (manager, HR, talent), application architecture design and iterative development of a React.js front end with Vite and Tailwind CSS. Integration uses Supabase and PostgreSQL for storage, authentication and APIs, n8n for process orchestration and the Llama model for decision support. Testing and validation phases refined the features against priority scenarios.

The platform turns business data into actionable insights through role-based dashboards, risk detection (overload, contract end dates, skill shortages), project viability analysis, a *what-if* simulator, a decision log, and structured HR request workflows. The embedded *Strategic Copilot* provides summaries, viability scores, recommendations, and a context-aware conversational assistant (project, team, alerts), powered by the **Llama** model and supported by automated orchestration flows.

The methodology combines requirements analysis, a modular multi-role architecture (manager, HR, and talent workspaces), and iterative development of a React front end (Vite, Tailwind CSS) connected to Supabase (PostgreSQL, authentication, API) and n8n for automation. Validation on critical user journeys—project steering, HR request handling, and talent self-service—confirms consistent alerting, useful recommendations, and a coherent multi-stakeholder experience.

Keywords : SaaS, Artificial Intelligence, Talent Management, Project Management, Decision Support.

Introduction Générale

La transformation numérique constitue aujourd'hui l'un des leviers majeurs de la compétitivité des organisations. Elle s'accompagne d'une volumétrie croissante de données RH et projet, produites par les outils de gestion, les feuilles de suivi et les échanges collaboratifs. Sans dispositif de consolidation, cette richesse informationnelle demeure difficilement exploitable au moment où une décision doit être prise. Les plateformes d'aide à la décision répondent précisément à cet écart, en transformant des données hétérogènes en indicateurs lisibles, en tendances observables et en recommandations susceptibles d'orienter l'action. Elle redéfinit les modes de production, de collaboration et de pilotage, en imposant une disponibilité permanente de l'information et une capacité d'adaptation accrue face à l'incertitude. Dans ce contexte, les outils d'aide à la décision occupent une place centrale : ils ne se limitent plus à restituer des données historiques, mais visent à contextualiser les signaux faibles, à mettre en évidence les écarts par rapport aux objectifs et à accompagner les décideurs dans le choix des actions les plus pertinentes. Pour les directions opérationnelles et les fonctions RH, cette évolution n'est pas un simple confort technologique ; elle répond à un besoin structurel de réactivité, de transparence et de coordination à l'échelle de l'entreprise.

La gestion des talents, des compétences, des charges de travail et du portefeuille de projets illustre particulièrement cette exigence. Les organisations doivent concilier la disponibilité des ressources, le développement des savoir-faire, la tenue des engagements contractuels et la réussite des livrables projet, dans un environnement où les priorités évoluent rapidement. Un retard sur un projet, une montée en charge imprévue ou un déficit de compétences sur une mission critique peuvent avoir des effets en cascade sur la performance collective. Par conséquent, disposer d'une vision consolidée, fiable et actualisée devient un prérequis pour anticiper les tensions, répartir équitablement la charge et aligner les décisions stratégiques avec la réalité du terrain.

Pourtant, les managers et les responsables RH rencontrent encore de nombreuses difficultés dans l'exercice quotidien de ce pilotage. Le manque de visibilité globale demeure une préoccupation récurrente : les informations sont souvent dispersées entre plusieurs outils, feuilles de calcul ou processus informels, ce qui complique la construction d'une lecture commune de la situation. La surcharge de travail constitue un second enjeu, lorsque certaines

ressources atteignent des seuils critiques sans alerte précoce ni mécanisme de rééquilibrage. Les fins de contrat, lorsqu'elles ne sont pas anticipées, fragilisent la continuité des projets et accroissent le risque de rupture de compétences. Les écarts entre compétences requises et compétences disponibles freinent le démarrage ou la poursuite des activités, tandis que la difficulté à prioriser les décisions — arbitrer entre plusieurs demandes RH, choisir un réaménagement d'équipe ou valider un plan de formation — retarde parfois des actions pourtant nécessaires. Ces contraintes, cumulées, illustrent la nécessité d'une approche intégrée plutôt que d'interventions ponctuelles et déconnectées.

L'intelligence artificielle et l'automatisation ouvrent des perspectives concrètes pour répondre à ces limites. L'analyse assistée des données permet de détecter plus tôt les situations à risque, de formuler des recommandations argumentées et de réduire le temps consacré aux tâches de synthèse répétitives. L'automatisation des workflows, quant à elle, structure les demandes, garantit une meilleure traçabilité des décisions et facilite le passage de l'information entre les acteurs concernés. Il ne s'agit pas de substituer le jugement humain, mais de le soutenir par des indicateurs explicites, des alertes contextualisées et des scénarios d'aide à l'arbitrage, afin que les décisions stratégiques reposent sur des éléments partagés et vérifiables.

C'est dans cette perspective qu'a été conçue la solution développée dans le cadre du présent projet de fin d'études : Strategic Copilot, plateforme SaaS intelligente d'aide à la décision pour la gestion stratégique des talents et des projets. Il s'agit d'un environnement web unifié destiné aux managers, aux équipes RH et aux talents, articulé autour d'espaces métier distincts mais cohérents. Pour les managers, la plateforme propose un pilotage du portefeuille projet, du suivi d'équipe, de la gestion des risques et des demandes adressées aux fonctions RH ou aux talents. Pour les responsables RH, elle couvre le référentiel des talents, le catalogue de compétences, l'identification des écarts critiques, les plans de formation, le traitement des demandes managers et les alertes organisationnelles. Pour les talents, elle offre une visibilité sur les missions, les tâches, la charge de travail, les compétences et les parcours de formation. Au cœur de l'expérience utilisateur, le Strategic Copilot intégré fournit des synthèses, des analyses de viabilité, des recommandations et un assistant conversationnel adapté au contexte métier consulté. Les tableaux de bord, les alertes, le simulateur de scénarios et le journal des décisions complètent ce dispositif en offrant aux utilisateurs une lecture immédiate de la situation, tout en conservant la trace des arbitrages effectués. L'ambition est ainsi de rapprocher, au sein d'un même environnement, la donnée brute, l'interprétation assistée et l'action concrète.

Les objectifs principaux du projet visent à traduire ces orientations en une solution opérationnelle et exploitable. Il s'agit tout d'abord de permettre la visualisation claire des indicateurs RH et projet, afin de disposer d'un tableau de bord partagé par rôle et par périmètre de responsabilité. Il s'agit ensuite de mettre en œuvre une détection proactive des risques,

notamment la surcharge, les échéances contractuelles et les manques de compétences, avec des mécanismes d’alerte intelligents. Le projet poursuit également l’analyse de la viabilité des projets, la génération de recommandations automatisées fondées sur l’intelligence artificielle, ainsi que la gestion structurée des demandes RH au sein de workflows décisionnels. Enfin, un objectif transversal consiste à offrir une expérience fluide, sécurisée et évolutive, capable d’accompagner la montée en charge fonctionnelle de la plateforme.

Sur le plan technique, la réalisation s’appuie sur une stack moderne adaptée aux exigences d’une application SaaS. L’interface utilisateur est développée en React.js [6], avec Vite [22] comme outil de build et Tailwind CSS [18] pour la conception visuelle et la cohérence des composants. La couche données et services s’appuie sur Supabase [17], qui fournit l’authentification, les API et l’accès à une base PostgreSQL [19] pour la persistance des informations métier. Les processus d’orchestration et d’intégration sont confiés à n8n [8], permettant d’automatiser les flux entre la plateforme, les services externes et les composants d’intelligence artificielle [9, 12]. L’enrichissement décisionnel repose sur des modèles de langage invoqués côté orchestrateur, mobilisés pour les recommandations, les synthèses et l’assistant conversationnel du copilote. Cette combinaison technologique a guidé les choix d’architecture retenus tout au long du projet.

La méthodologie générale adoptée suit une démarche progressive, structurée en phases complémentaires. Une première phase d’analyse a permis d’identifier les besoins, les acteurs, les contraintes et les cas d’usage prioritaires associés aux espaces manager, RH et talent. Une phase de conception a ensuite défini l’architecture applicative, les principaux modules fonctionnels, les flux de données et les parcours utilisateurs. La phase de réalisation a été menée de manière itérative, avec des livraisons successives des écrans, des intégrations API et des mécanismes d’alerte et de recommandation. Enfin, des campagnes de tests et de validation ont été conduites sur les scénarios critiques — pilotage projet, traitement des demandes RH, suivi talent — afin de vérifier la cohérence fonctionnelle, la stabilité technique et l’apport réel de l’aide à la décision. Cette organisation a favorisé un alignement continu entre les objectifs métier et les choix d’implémentation. Chaque itération a été l’occasion de confronter les prototypes aux attentes des utilisateurs cibles, d’ajuster les écrans de pilotage et de renforcer la fiabilité des échanges entre le front-end, les services Supabase et les flux automatisés orchestrés par n8n. Cette rigueur méthodologique a permis de maintenir une cohérence d’ensemble, malgré la diversité des fonctionnalités à couvrir.

Le présent rapport retrace l’ensemble de cette démarche et en présente les apports de manière structurée. Le premier chapitre est consacré au contexte général du projet et à l’analyse des besoins. Le deuxième chapitre présente l’étude bibliographique : fondements RH, gestion des talents, pilotage de portefeuille, aide à la décision, automatisation et technologies web mobilisées par Strategic Copilot. Le troisième chapitre formalise le cahier des charges, les spécifications et la planification. Le quatrième chapitre présente la conception fonctionnelle

et technique complète du système. Le cinquième chapitre détaille la réalisation et l'implémentation : environnement de développement, front-end, workflows n8n, base de données et interfaces livrées. Le sixième chapitre présente les tests réalisés, les résultats obtenus et l'évaluation du système. La conclusion générale revient sur les enseignements du projet, ses limites et les perspectives d'évolution envisageables pour consolider la plateforme comme outil durable de pilotage stratégique des talents et des projets.

Chapitre 1

Contexte général du projet et analyse des besoins

1.1 Introduction

Ce chapitre ouvre le rapport en présentant le cadre général du projet et en précisant les besoins auxquels répond la solution développée pendant le stage. Après l'introduction générale, qui a introduit les grands thèmes liés à la digitalisation des ressources humaines et à l'aide à la décision, il s'agit ici d'entrer dans le concret : le contexte de l'entreprise d'accueil, la problématique rencontrée, les objectifs visés et les fonctionnalités attendues de la plateforme. L'objectif est de donner au lecteur une vision claire avant d'aborder, dans les chapitres suivants, l'étude bibliographique, le cahier des charges, la conception, la réalisation et les tests.

Le projet porte sur **Strategic Copilot**, une plateforme web d'aide à la décision pour le pilotage des projets et des talents. Elle s'inscrit dans l'offre plus large **Augmented Talents** et s'adresse à trois types d'utilisateurs : les **managers**, qui suivent leur portefeuille de projets et leur équipe; les **responsables RH**, qui gèrent les compétences, les formations et les demandes des managers; et les **talents**, qui consultent leurs missions, leurs tâches et leur charge de travail. J'ai conçu et développé l'application **copilot-ui** de manière à regrouper ces trois espaces dans une même solution, avec des droits d'accès adaptés à chaque profil. Le développement a débuté par l'**espace Manager**, aujourd'hui le plus abouti en termes de fonctionnalités, avant d'étendre progressivement les modules RH et Talent; l'ensemble de la plateforme reste néanmoins couvert par le même socle technique.

Pour rédiger cette analyse, je me suis appuyée sur le document *PFE Agentic AI Strategic Talents*, sur le cahier des charges fourni en entreprise, sur les échanges avec les encadrants, et sur l'étude du code source du dépôt **frontend-ui**. Seules les fonctionnalités réellement

présentes ou clairement prévues dans ces documents y sont mentionnées. Ce chapitre pose ainsi les bases nécessaires pour comprendre la suite du rapport et pour relier chaque besoin métier aux modules concrets de la plateforme.

1.2 Présentation de l'entreprise d'accueil

Le stage de fin d'études s'est déroulé au sein de **AphelionX Innovations**, entreprise marocaine basée à Tétouan, spécialisée dans le développement logiciel, le conseil en systèmes d'information, l'automatisation des processus métier fondée sur l'intelligence artificielle et l'accompagnement des organisations dans leur transformation digitale. Ce positionnement place l'entreprise à l'intersection de l'ingénierie applicative et de l'innovation par les agents intelligents, ce qui constitue un cadre particulièrement adapté à un projet tel que Strategic Copilot, où la valeur repose autant sur la qualité de l'interface que sur la fiabilité des workflows backend et des modèles d'analyse.

Dans ce contexte, AphelionX développe la solution **Strategique Copilot**, plateforme modulaire orientée vers le pilotage conjoint des projets et des compétences. Strategic Copilot en est la brique d'intelligence décisionnelle : elle ne se substitue pas aux outils de gestion opérationnelle classiques, mais les prolonge en consolidant des signaux projet et RH, en détectant des risques et en proposant des recommandations que les décideurs peuvent examiner, accepter ou rejeter. L'infrastructure technique que j'ai mise en place repose sur une instance n8n dédiée (`n8nprod.aphelionxinnovations.com`), qui expose l'ensemble des webhooks métier et orchestre les workflows agentiques, ainsi que sur une base de données PostgreSQL hébergée via **Supabase** pour la persistance relationnelle et, selon les cas, le stockage de fichiers (rapports PDF, avatars). Le front-end React consomme exclusivement ces services via des appels HTTP sécurisés ; il n'effectue pas de calculs critiques de scores de viabilité, de risques ou de recommandations en local, conformément au principe API-first rappelé dans le typage métier (`types/crud-domain.ts`).

L'encadrement en entreprise a été assuré par Monsieur **Hicham Taoufik**, Monsieur **Youssef Moufak**, Monsieur **Hamza Sadouk** et Monsieur **Mouhy Eddine Saadani**, qui ont orienté les choix fonctionnels, validé les priorités de livraison et accompagné l'intégration des composants Copilot et des agents métier documentés dans le cahier Agent AI. L'encadrement pédagogique de Madame **Soundouss Abroun**, au sein de l'École Marocaine des Sciences de l'Ingénieur (EMSI) de Tanger, a garanti la rigueur méthodologique, la structuration du rapport et l'alignement du livrable sur les exigences académiques du diplôme d'Ingénieur d'État en Informatique et Réseaux, option MIAGE. Le projet s'inscrit ainsi à l'intersection d'une démarche industrielle de type produit SaaS et d'une démarche d'ingénierie logicielle structurée, où la traçabilité des décisions et la clarté des interfaces sont considérées comme

des exigences au même titre que la performance technique.

1.3 Contexte du stage

Le stage s’est déroulé du **2 février 2026 au 2 juillet 2026**, soit une durée de **cinq mois**, dans le cadre du Projet de Fin d’Études de l’EMSI Tanger en vue de l’obtention du diplôme d’Ingénieur d’État en Informatique et Réseaux, option MIAGE. J’ai assuré personnellement l’ensemble des grandes étapes du projet Strategic Copilot : analyse des besoins à partir du document *PFE Agentic AI Strategic Talents*, conception de l’architecture fonctionnelle et technique, développement de l’interface avec **React**, **Vite** et **TypeScript**, mise en place des workflows backend avec **n8n**, modélisation et alimentation de la base **Supabase PostgreSQL**, intégration des modèles **Llama** pour les fonctions d’intelligence artificielle, puis tests et validation sur l’environnement de préproduction.

Sur le plan académique, ce stage représente la synthèse des compétences acquises en conception, en développement web, en intégration de services et en rédaction technique. Sur le plan professionnel, il répond au besoin d’AphelionX de disposer d’une plateforme unifiée, multilingue et sécurisée, capable de restituer les analyses produites par les agents métier sans les déformer. J’ai développé les modules des trois espaces — **Manager**, **RH** et **Talent** — au sein d’une même application routée dans `main.tsx`. L’espace Manager (tableau de bord, portefeuille projet, Mission Control, équipe, risques, demandes RH, rapports, notifications, Copilot et Helper) a été structuré en premier et reste aujourd’hui le plus riche fonctionnellement ; les espaces RH et Talent disposent pour leur part de parcours complets (tableaux de bord, référentiels, demandes, missions, charge, compétences, formations), intégrés aux mêmes services d’authentification et aux mêmes workflows n8n.

Le déroulement du projet a suivi une logique incrémentale. Chaque livraison a été validée par des tests d’intégration contre les webhooks n8n, par des ajustements des contrats TypeScript (`api.types.ts`) et par des retours des encadrants sur l’ergonomie des écrans. L’environnement local repose sur **Vite** avec un proxy vers n8n pour contourner les contraintes CORS. Les rôles, les statuts utilisateurs et l’authentification JWT sont entièrement gérés côté serveur ; le front-end n’ouvre pas de parcours d’auto-inscription, le provisionnement des comptes relevant du périmètre RH.

1.4 Problématique

La problématique métier, formulée dans le document *PFE Agentic AI Strategic Talents* et confirmée par les retours recueillis durant le stage, peut être analysée comme une tension

structurelle entre la **richesse des données** disponibles en entreprise et la **pauvreté de leur mise en scène décisionnelle**. Dans de nombreuses organisations, les informations relatives aux projets (jalons, budgets, affectations), aux talents (compétences, contrats, charge), aux demandes RH (formation, mobilité, recrutement, réallocation) et aux signaux de risque (surcharge, retards, écarts de compétences) sont produites par des outils hétérogènes : feuilles de calcul, messageries, tickets, systèmes RH partiellement connectés. Cette fragmentation limite la visibilité globale, retarde la détection des dérives et rend la prise de décision plus coûteuse en temps et en coordination.

Pour le **manager**, la difficulté consiste à piloter simultanément un portefeuille de projets et une équipe de collaborateurs sans disposer d'une vue consolidée actualisée. Les retards sur un livrable, une montée en charge imprévue sur un profil clé ou l'approche d'une fin de contrat peuvent ne être identifiés que lorsque les marges de manœuvre sont réduites. Pour le **responsable RH**, la problématique est différente mais connexe : il doit maintenir un référentiel de compétences fiable, détecter les écarts critiques (*skill gaps*), instruire des plans de formation et de mobilité, tout en traitant des demandes émises par les managers avec des types métier variés (*skill_gap*, *reallocation*, *training*, *overload*, *recruitment*) sans toujours disposer d'une vision temps réel de l'impact sur les projets en cours. Pour le **talent**, l'enjeu est la transparence : comprendre ses missions, ses tâches, sa charge de travail et ses axes de développement, sans accéder à des données hors périmètre ni subir une opacité sur les décisions qui le concernent.

À ces difficultés s'ajoute l'émergence de l'**intelligence artificielle générative** comme levier d'analyse et de synthèse. Les modèles de langage, lorsqu'ils sont correctement encapsulés dans des workflows contrôlés (orchestration n8n, paramètre `use_ai`, modèles tels que **Llama** mobilisés côté serveur selon le cadrage du projet), peuvent accélérer la détection de signaux faibles et la formulation de recommandations. Toutefois, leur adoption en contexte managérial et RH exige une **explicabilité** des sorties, une séparation claire entre corrélation et décision, et une traçabilité des arbitrages humains. Strategic Copilot ne prétend pas remplacer le jugement des décideurs ; il vise une aide à la décision *augmentée*, en transformant des analyses multi-agents complexes en indicateurs, alertes, scores de viabilité et propositions d'action que l'utilisateur peut valider, modifier ou rejeter.

La problématique du stage peut donc se résumer ainsi : comment concevoir et livrer une interface web unifiée, sécurisée par rôle, qui centralise les données RH et projet, expose les résultats des agents d'analyse et de risque, facilite la collaboration manager–RH–talent, et respecte une architecture où le calcul métier critique reste côté serveur ? Cette question oriente l'ensemble des objectifs et des besoins formalisés dans les sections suivantes.

1.5 Objectifs du projet

Les objectifs du projet de fin d'études découlent directement de la problématique et du cahier des charges Agentic AI. Ils se déclinent en quatre dimensions complémentaires : métier, technique, agentique et utilisateur.

Sur le plan **métier**, il s'agit de **centraliser** les données relatives aux projets et aux talents dans une expérience cohérente, de **détecter** proactivement les risques liés aux surcharges, aux retards, aux écarts de compétences et aux échéances contractuelles, de **fournir** des recommandations issues de l'intelligence artificielle de manière structurée et interprétable, et de **faciliter** la collaboration entre managers, responsables RH et talents via des flux de demandes, d'alertes et de notifications traçables. La vision produit affichée dans le dépôt résume cette ambition : transformer des analyses complexes portant sur les projets, les compétences, les charges et les risques en décisions managériales claires, explicables et directement actionnables.

Sur le plan **technique**, l'objectif est de livrer une application **React 19** typée en **TypeScript**, construite avec **Vite 7**, stylée en **Tailwind CSS 4** et fondée sur des composants accessibles (**React Aria Components**, kit **Untitled UI**), consommant les webhooks **n8n** et la base **PostgreSQL** via Supabase sans logique décisionnelle critique dans le navigateur. L'authentification repose sur des jetons **JWT** (`/webhook/login`, `/webhook/refresh`); les routes applicatives sont protégées par rôle (`ProtectedRoute`). L'internationalisation (**i18next**, français, anglais, arabe) et la gestion d'état serveur (**TanStack React Query**) visent une expérience fluide malgré la latence potentielle des workflows longs.

Sur le plan **agentique**, l'objectif est d'exposer de manière homogène les résultats des agents documentés : **Project Analysis Agent** (`WF_Project_Analysis`, analyste projet), **Talent Matching Agent** (`WF_Talent_Matching`, appariement compétences-besoins), **Risk & Weak Signal Agent** (`WF_Project_Risk`, surveillance et scan Watchdog), et **Strategic Orchestrator Agent** (`WF_Project_Viability`, viabilité et scénarios). Le Helper conversationnel (`POST /webhook/api/helper/chat`) et le Copilot global (`CopilotProvider`, `CopilotPanel`) constituent les points d'interaction naturels avec ces capacités.

Sur le plan **utilisateur**, l'objectif est de proposer trois espaces routés (`/workspace/manager`, `/workspace/rh`, `/workspace/talent`), chacun adapté aux décisions de son profil, avec des tableaux de bord analytiques, des parcours de traitement des alertes et des demandes, et des mécanismes de reporting (génération PDF, envoi par e-mail via `/webhook/reports/send-email` lorsque le workflow est disponible). Les critères de réussite incluent la disponibilité des parcours critiques testés en préproduction : connexion, pilotage projet, scan Watchdog, demande RH, simulation what-if, enregistrement d'une décision dans le journal Copilot, et cohérence des données affichées avec les réponses des workflows n8n.

1.6 Présentation générale de Strategic Copilot

1.6.1 Vision et positionnement

Strategic Copilot se définit comme une plateforme **SaaS intelligente** de pilotage stratégique des talents et des projets. Son positionnement au sein d'Augmented Talents est celui d'un **copilote décisionnel** : il ne remplace ni l'outil de gestion de projet classique ni le système d'information RH, mais les enrichit en agrégeant des indicateurs, en détectant des risques et en proposant des scénarios d'arbitrage. La vision, rappelée dans le README du dépôt, insiste sur la conversion d'analyses multi-agents en décisions actionnables (*Transform complex multi-agent analysis into clear, actionable decisions*).

Le cœur métier du pilotage projet repose sur un **score de viabilité** et une **décision synthétique** produits côté orchestrateur n8n, selon une pondération documentée dans le cahier des charges : Skills Fit (40 %), Capacité (35 %), Budget (25 %). Le score est traduit en recommandation Continue (score $\geq 7,5$), Adjust (entre 4 et 7,5) ou Stop (< 4). L'interface se limite à afficher les champs `viability_score`, `risk_score` et `decision` renvoyés par le serveur, ce qui garantit une cohérence entre les utilisateurs et évite les divergences de calcul. Le positionnement « SaaS » implique par ailleurs une séparation nette entre la couche présentation (build Vite, déploiement possible sur Vercel avec relais des webhooks) et la couche orchestration–données (n8n, PostgreSQL), chaque couche pouvant évoluer indépendamment tant que les contrats d'API restent stables.

1.6.2 Les trois espaces utilisateurs

L'application structure l'expérience en trois espaces utilisateurs distincts mais articulés autour des mêmes entités métier (projets, talents, compétences, alertes, demandes). Le routage est centralisé dans `main.tsx` ; l'accès est filtré par rôle JWT via `ProtectedRoute` et des layouts dédiés qui assurent une navigation homogène au sein de chaque perspective.

Espace Manager. L'espace `/workspace/manager` concentre les besoins du pilotage opérationnel et stratégique. Le **tableau de bord** (`DashboardPage`) restitue des indicateurs consolidés et, lorsque le backend les fournit, des blocs issus des agents Matchmaker et Analyst. Le **portefeuille projets** (`ProjectsPage`) permet de parcourir l'ensemble des projets suivis et d'ouvrir le modal **Mission Control** (`ProjectMissionControlModal`), véritable centre de pilotage à 360 degrés d'un projet : synthèse, équipe, tâches, risques, simulation what-if et journal des décisions liées. La gestion d'**équipe** (`TeamPage`, `TalentDetailPage`) offre une lecture fine des profils, des allocations et des alertes, avec possibilité de déclencher un scan Watchdog (POST `/webhook/api/watchdog/scan`). La page **Risques & alertes** (`RisksPage`)

propose une vue transverse des signaux critiques, complétée par la page **Notifications** (NotificationsPage) qui consomme les workflows de type WF_Manager_Notifications. Les **demandes RH** (ManagerRhRequestsPage, composants rh-requests) et les **demandes talent** (TalentRequestsPage) matérialisent les flux collaboratifs vers les autres acteurs. Le **journal des décisions** (DecisionLogPage), les **rapports** (ReportsPage, génération board-pack, historique, partage par e-mail) et le **Helper** (HelperChatPage) complètent l'espace. Le **Copilot** contextuel (CopilotProvider, panneau global CopilotPanel) accompagne l'utilisateur selon le contexte consulté (tableau de bord, détail projet, staffing). Cet espace concentre aujourd'hui le plus grand nombre de fonctionnalités avancées; il a constitué le point de départ du développement, sans exclure pour autant la réalisation des modules RH et Talent décrits ci-dessous.

Espace RH. L'espace /workspace/rh répond aux besoins de gouvernance des ressources humaines et de traitement des demandes managers. Le **tableau de bord RH** (RhDashboardPage) offre une vue agrégée des indicateurs RH. Le module **employés / talents** (RhEmployeesPage) permet de consulter et d'administrer le référentiel des collaborateurs. Le **catalogue de compétences** (RhSkillsCatalogPage) et la page des **écarts critiques** (RhCriticalGapsPage) soutiennent la politique de compétences. Les **plans de formation** (RhTrainingPlansPage) et la **mobilité interne** (RhMobilityPage) adressent le développement et la circulation des talents. Les **alertes organisationnelles** (RhOrgAlertsPage) complètent la surveillance transverse. Le traitement des **demandes managers** (RhManagerRequestsPage, rh-manager-requests-page) permet d'accepter, rejeter ou faire progresser les demandes typées. Des vues **comptes et sessions** (RhAccountsWorkspacePage, RhSessionsWorkspacePage) assurent l'administration des accès, tandis que les **rapports RH** (RhReportsWorkspacePage) et une vue **projets** (ProjectsPage dans le layout RH) maintiennent l'alignement avec le portefeuille manager. J'ai également développé cet espace : tableaux de bord, pages métiers et appels API dédiés (/workspace/rh/*) sont opérationnels et connectés aux workflows n8n correspondants.

Espace Talent. L'espace /workspace/talent adopte le point de vue du collaborateur. Le **tableau de bord personnel** (TalentDashboardPage) synthétise la situation individuelle. Les pages **Mes projets** (TalentProjectsPage), **Mes tâches** (TalentTasksPage) et **Charge de travail** (TalentWorkloadPage) rendent visibles les affectations et la charge associée. La page **Compétences** (TalentSkillsPage) et celle des **Formations** (TalentTrainingPage) soutiennent le développement professionnel. Les **notifications** (TalentNotificationsPage) et le **profil** (TalentProfileWorkspacePage) ferment la boucle en offrant une lecture individuelle des informations qui, en amont, alimentent les analyses vues par le manager et le RH. Les données talent sont chargées via des endpoints agrégés de type /api/workspace/talent/*, ce qui garantit un périmètre strict au collaborateur connecté.

La coexistence de ces trois espaces dans une même application n'est pas une simple juxtaposition d'écrans : elle traduit un modèle organisationnel où les décisions se prennent en chaîne. Le manager identifie un risque ou un besoin, formalise une demande RH, le responsable RH instruit et met à jour le référentiel, le talent constate l'impact sur ses missions et sa charge. Strategic Copilot se veut le médiateur numérique de cette chaîne.

1.6.3 Fonctionnalités transverses

Plusieurs capacités traversent les trois espaces et constituent l'identité produit de Strategic Copilot au-delà des menus spécifiques à chaque rôle.

L'**authentification sécurisée par rôle** (AuthProvider, API `auth.api.ts`) conditionne l'accès aux workspaces et évite les fuites de données entre profils. Les **tableaux de bord analytiques**, appuyés sur Recharts pour certaines visualisations manager, traduisent des agrégats serveur en graphiques et cartes KPI lisibles. La **détection automatique des risques** et le mécanisme d'**alertes et notifications** s'appuient sur les workflows Watchdog et Manager Notifications ; l'interface permet l'acquiescement, la résolution ou l'ignorance selon que l'alerte est liée à une entrée `risk_alerts` ou à une notification seule.

Les **recommandations IA** sont restituées via le Copilot, le panneau Matchmaker, les synthèses Analyst et les réponses du Strategist lorsque `use_ai` est activé dans les appels n8n. Le **simulateur what-if** (WhatIfProvider, POST `/webhook/api/project/what-if`) permet de tester l'impact de modifications d'allocation, de délai ou de compétences avant arbitrage. La **génération de rapports PDF** et l'**envoi par e-mail** (`use-reports-n8n`, `sendReportEmail`) supportent la gouvernance et le partage formel des analyses. Le **journal des décisions** (DecisionLogPage, API `/webhook/manager/copilot-decisions`) assure la traçabilité des choix Continue / Adjust / Stop et des motifs associés. Enfin, le **Helper conversationnel** propose un dialogue contextualisé avec suggestions d'actions (`assign`, `training`, `review`, `risk`), complétant le Copilot plus orienté synthèse et scores.

Ces fonctions transverses incarnent l'architecture multi-agents : le front-end orchestre les appels et la restitution, tandis que n8n enchaîne lecture PostgreSQL, règles métier et inférences Llama sans exposer les secrets de modèle au client.

1.7 Analyse des besoins

1.7.1 Acteurs et cas d'usage

L'analyse des besoins commence par l'identification des acteurs et de leurs intentions principales, telles qu'elles ressortent du cahier Agent AI et des parcours implémentés.

Le **manager** souhaite connaître l'état de son portefeuille projet, identifier les projets à risque, affecter ou réaffecter des talents, formuler des demandes RH ou talent, simuler des scénarios avant engagement, consigner ses décisions et consulter des rapports de synthèse. Ses cas d'usage prioritaires incluent l'ouverture du Mission Control sur un projet critique, le déclenchement d'un scan Watchdog sur un collaborateur en surcharge, la création d'une demande de type **training** ou **reallocation**, et l'exécution d'une simulation what-if avant de valider un arbitrage Strategist.

Le **responsable RH** souhaite maintenir le référentiel talents et compétences, détecter les écarts critiques, planifier formations et mobilités, traiter les demandes managers dans des délais maîtrisés, surveiller les alertes organisationnelles et administrer les comptes utilisateurs. Ses cas d'usage couvrent l'instruction d'une demande manager (acceptation, rejet, mise en progression), la consultation du catalogue de compétences, l'analyse des gaps sur un périmètre métier et la génération de rapports RH.

Le **talent** souhaite consulter ses missions, mettre à jour l'état de ses tâches lorsque le workflow le permet, suivre sa charge, visualiser ses compétences et formations, recevoir des notifications pertinentes et gérer son profil. Le système doit garantir qu'il n'accède pas aux données d'autres collaborateurs ni aux vues stratégiques réservées au manager.

Les processus transverses documentés (P0 à P5 dans le cahier Agentic AI) structurent ces cas d'usage : synchronisation des données (**WF_Data_Sync**), analyse projet, matching talents, analyse des risques, orchestration stratégique et simulation. Chaque processus se traduit par un ou plusieurs webhooks HTTP déclenchés depuis l'interface ou par des tâches planifiées côté n8n.

1.7.2 Besoins fonctionnels

Les besoins fonctionnels expriment les services attendus du système. Ils ont été validés par confrontation entre le cahier des charges et les modules présents dans **copilot-ui**.

Le système doit assurer l'**authentification et la gestion de session** (connexion, renouvellement, déconnexion, profil, changement de mot de passe). Il doit permettre le **pilote projet** : liste, détail versionné (**wmp-detail-v1**), affectations (**wmp-assign-v1**, **wmp-unassign-v1**), tâches (**wmt-*-v1**), représentation du cycle de vie (**ProjectLifecycleStepper**). Il doit couvrir la **gestion des talents et compétences** côté RH et manager : référentiel, fiche talent, catalogue Skill, liaisons TalentSkill. Il doit exposer les **risques et alertes** : liste par projet, notifications manager, actions de résolution ou d'acquittement. Il doit délivrer les **recommandations IA** via Copilot, Matchmaker, Analyst, Helper et Strategist. Il doit gérer les **demandes RH** (création manager, liste, traitement RH par PATCH). Il doit proposer la **simulation what-if** et les **rapports** (génération, historique, envoi e-mail). Il doit enfin offrir

les **espaces talent** (dashboard, projets, tâches, charge, compétences, formations, notifications, profil).

Le tableau 1.1 synthétise ces besoins par domaine et indique la trace dans l'application livrée.

TABLE 1.1 – Besoins fonctionnels et trace dans l'application

Domaine	Besoin	Réalisation (copilot-ui)
Authentification	Connexion sécurisée par rôle	AuthProvider, /webhook/login
Portefeuille	Vue consolidée projets	DashboardPage, dashboard manager
Projet	Pilotage détaillé 360°	ProjectMissionControlModal
Équipe	Suivi talents, scan risques	TeamPage, Watchdog
RH	Référentiel, écarts, demandes	Routes /workspace/rh/*
Talent	Missions, charge, compétences	Routes /workspace/talent/*
IA	Synthèse, chat, arbitrage	CopilotProvider, Helper, Strategist
Simulation	Scénarios what-if	WhatIfProvider
Reporting	PDF, e-mail, historique	ReportsPage, reports.api
Notifications	Alertes, acquittement	NotificationsPage

1.7.3 Besoins non fonctionnels

Les besoins non fonctionnels encadrent la qualité de la solution et conditionnent son adoption en entreprise.

En matière de **performance**, l'application doit tolérer la latence des workflows n8n grâce à des délais configurables (VITE_HTTP_TIMEOUT_MS, VITE_PORTFOLIO_TIMEOUT_MS) et à TanStack React Query (cache, invalidation ciblée après mutations, par exemple invalidateManagerRiskQuery). Les états de chargement, d'erreur et de vide doivent être explicites pour éviter toute ambiguïté sur l'état du système.

En matière de **sécurité**, les jetons JWT ne doivent pas exposer les clés des modèles IA ; les routes sont filtrées par rôle ; le client gère le renouvellement de session sur réponse 401. Les données talent ne sont accessibles qu'au talent connecté via les endpoints workspace dédiés.

En matière de **fiabilité**, les parseurs tolèrent les réponses n8n enveloppées (json, body, tableaux notifications ou items) ; les messages d'erreur sont orientés utilisateur (readUserFacingApiResponse).

En matière de **maintenabilité**, TypeScript structure les contrats dans api.types.ts ;

les chemins d'API sont externalisés en variables `VITE_*` pour accompagner l'évolution des webhooks versionnés (`wmp-detail-v1`, etc.).

En matière d'**utilisabilité**, l'interface est responsive, accessible (React Aria), traduite (i18next) et cohérente visuellement (Tailwind CSS 4, composants Untitled UI). En matière d'**explicabilité IA**, les scores et décisions affichés proviennent du serveur ; l'interface rappelle, sur certains panneaux, que les calculs ne sont pas refaits côté client.

En matière de **traçabilité**, le journal des décisions, l'historique des rapports et le suivi des demandes RH permettent un audit a posteriori des choix organisationnels.

1.7.4 Contraintes identifiées

Plusieurs contraintes ont guidé les choix de conception et expliquent certains arbitrages du stage.

La contrainte de **temps limité** (cinq mois) a conduit à organiser le développement en commençant par l'espace Manager et les parcours les plus critiques pour la démonstration, tout en livrant les espaces RH et Talent sur la même architecture applicative.

La **complexité des processus RH** (typologie des demandes, SLA, validation multi-niveaux) exige des workflows n8n maintenables et des contrats d'API stables ; toute évolution métier doit être répercutée côté serveur avant l'interface.

La contrainte d'**explicabilité des recommandations IA** interdit de présenter des sorties de modèle comme des vérités définitives ; l'interface doit laisser la responsabilité au décideur et journaliser les arbitrages.

La **performance des workflows n8n** (analyses longues, appels Llama) impose des timeouts élevés et des indicateurs de progression ; le front-end ne doit pas bloquer l'expérience sans retour visuel.

La contrainte de **cohérence et qualité des données** rappelle que la valeur de Strategic Copilot dépend de l'exhaustivité des enregistrements PostgreSQL (affectations, compétences, dates de contrat) ; une alerte sans `risk_alert_id` correctement renseigné ne peut pas être résolue via le PATCH `risk-alerts` et doit être traitée par acquittement notification, comportement que l'interface gère désormais explicitement.

Enfin, les contraintes architecturales **API-first** et **pas de calcul front** (`crud-domain.ts`) imposent que toute nouvelle règle de viabilité, de matching ou de risque transite par n8n avant affichage. Ces contraintes, loin de freiner le projet, ont permis de maintenir une séparation claire des responsabilités et de préparer les évolutions backend sans remise en cause systématique des écrans.

1.8 Conclusion

Ce chapitre a présenté le contexte général du projet Strategic Copilot et l'analyse des besoins qui ont guidé mon travail de fin d'études. L'entreprise d'accueil AphelionX Innovations, le cadre du stage à l'EMSI Tanger et l'encadrement professionnel et pédagogique ont été rappelés. La problématique de fragmentation des données, de détection tardive des dérives et d'arbitrages peu traçables a été reliée aux objectifs du livrable.

La présentation de la plateforme a montré comment Strategic Copilot réunit trois espaces complémentaires — **Manager**, **RH** et **Talent** — autour d'une architecture multi-agents (analyse projet, matching talents, risques, orchestrateur stratégique), d'une stack React–n8n–PostgreSQL–Supabase et d'une intelligence artificielle orchestrée côté serveur (modèles Llama). J'ai personnellement porté l'ensemble du cycle projet, de l'analyse des besoins à la validation, en commençant par l'espace Manager, le plus avancé à ce stade, sans que mon travail se limite à ce seul périmètre.

L'analyse des besoins a formalisé les attentes des acteurs, les besoins fonctionnels et non fonctionnels, ainsi que les contraintes observées dans le code. Les chapitres suivants détailleront l'étude bibliographique, le cahier des charges, la conception, la réalisation et les tests de la solution que j'ai développée.

Chapitre 2

Étude Bibliographique

2.1 Introduction

Le présent chapitre expose les fondements théoriques, méthodologiques et technologiques sur lesquels s'appuie Strategic Copilot, plateforme SaaS d'aide à la décision pour la gestion stratégique des talents et des projets. Il fait suite au chapitre consacré au contexte général et à l'analyse des besoins, où ont été identifiés les acteurs, les tensions métier et les exigences traduites dans l'application *copilot-ui*. L'objectif n'est pas de dresser un inventaire encyclopédique des domaines concernés, mais de construire un socle conceptuel permettant de justifier les choix fonctionnels et techniques du projet, en les reliant systématiquement aux capacités observables dans le code source et dans le document de cadrage *PFE Agentic AI Strategic Talents*.

La démarche adoptée consiste à enchaîner une revue des grands courants de la transformation digitale des ressources humaines, de la gestion des talents et des compétences, du pilotage de portefeuille projet, de l'intelligence artificielle appliquée à l'aide à la décision, de l'automatisation des processus et des technologies web contemporaines. Une section de synthèse positionne ensuite Strategic Copilot au croisement de ces courants, avant de conclure sur les apports de cette étude pour les chapitres suivants consacrés au cahier des charges, à la réalisation et aux tests.

Il convient de préciser la portée de cette revue. Les références mobilisées relèvent de la littérature académique en systèmes d'information et en management des ressources humaines [16, 5], des référentiels professionnels en gestion de projet [10] et des documentations techniques des outils effectivement utilisés [6, 22, 19, 8, 9]. Lorsqu'un concept est appliqué à Strategic Copilot, l'affirmation est vérifiable dans le dépôt *frontend-ui* ou dans le document *PFE Agentic AI Strategic Talents*. Cette discipline évite de sur-vendre des capacités absentes du produit livré tout en donnant au lecteur les clés pour comprendre pourquoi telle

architecture a été retenue plutôt qu’une autre.

2.2 Transformation digitale des ressources humaines

2.2.1 Enjeux de la digitalisation RH

La transformation digitale des fonctions RH [16] désigne l’ensemble des mutations induites par l’adoption massive des outils numériques dans la gestion administrative, le développement des compétences, la relation aux collaborateurs et le pilotage de la performance. Les travaux de la fonction RH et des cabinets de conseil convergent sur plusieurs constats. Les organisations produisent davantage de données sur les parcours, les compétences, les absences et les charges, mais ces données restent souvent cloisonnées dans des systèmes hétérogènes. Les décideurs opérationnels, notamment les managers de proximité, peinent à disposer d’une vision consolidée au moment de l’arbitrage. La réactivité attendue face aux tensions de charge, aux échéances contractuelles ou aux écarts de compétences exige des indicateurs actualisés et partagés, et non des extractions manuelles périodiques.

Pour Strategic Copilot, ces enjeux se traduisent par la nécessité de rapprocher les données RH et les données projet dans un même environnement décisionnel. La plateforme ne se limite pas à un SIRH administratif : elle vise un pilotage transversal où le manager visualise simultanément la viabilité d’un projet, le niveau de risque associé et l’état de son équipe, tandis que le responsable RH traite des demandes structurées (`skill_gap`, `reallocation`, `training`, `overload`, `recruitment`) issues des analyses agentiques. Cette orientation répond directement au constat de fragmentation relevé dans le chapitre précédent et matérialisé dans le dépôt `frontend-ui` par trois espaces utilisateurs distincts mais alimentés par les mêmes référentiels PostgreSQL.

La digitalisation RH impose également des exigences de traçabilité et de gouvernance. Lorsqu’une recommandation d’IA suggère une formation, une réaffectation ou un report de périmètre, l’organisation doit pouvoir retrouver le contexte de la décision, les scores ayant motivé l’alerte et la validation humaine éventuelle. Strategic Copilot répond à cette exigence par le journal des décisions (`DecisionLogPage`), l’historique des demandes RH et la persistance des résultats analytiques dans des tables dédiées (`project_viability_scores`, `ai_decisions_log`), sans recalcul opaque côté navigateur, conformément au principe inscrit dans `crud-domain.ts`.

2.2.2 Systèmes d'information RH modernes

Les systèmes d'information RH modernes (SIRH) couvrent traditionnellement la gestion administrative du personnel, la paie, le temps et les compétences. Leur évolution récente s'oriente vers des plateformes intégrées, souvent délivrées en mode SaaS, capables d'échanger avec des outils projet, financiers ou collaboratifs. L'architecture type repose sur une base relationnelle centralisée, des API exposant les entités métier et des tableaux de bord par rôle. La tendance est à la modularité : plutôt qu'un monolithe unique, l'entreprise assemble des briques spécialisées reliées par des bus d'intégration ou des orchestrateurs de workflows.

Strategic Copilot s'inscrit dans cette logique modulaire au sein de l'écosystème **Augmented Talents**. Le front-end React [6] consomme des services exposés par n8n [8] ; PostgreSQL [19], hébergé via Supabase [17], constitue la source de vérité des entités `talents`, `skills`, `talent_skills`, `workload_snapshots` et des objets projet. L'authentification et le contrôle d'accès par rôle (`UserRole : manager, rh, talent`) reproduisent le modèle classique des SIRH modernes, enrichi d'une couche décisionnelle agentique absente des outils purement administratifs. Le responsable RH accède ainsi à un tableau de bord (`RhDashboardPage`), à un catalogue de compétences et à des écrans d'écarts critiques, tandis que le talent consulte ses missions et sa charge sans accéder aux fonctions de pilotage stratégique réservées au manager.

L'originalité du projet réside dans le fait que le SIRH étendu n'effectue pas lui-même les calculs de viabilité ou de matching : ils sont délégués à des workflows n8n spécialisés, ce qui préfigure une architecture « best-of-breed » où l'interface unifie l'expérience utilisateur et l'orchestrateur garantit la cohérence analytique. Cette séparation entre présentation et moteur décisionnel correspond aux recommandations architecturales des systèmes d'information contemporains, fondées sur des API stables et une persistance relationnelle robuste.

Par ailleurs, les SIRH modernes intègrent de plus en plus l'expérience collaborateur et la mobilité interne. Les écrans RH de Strategic Copilot — mobilité, plans de formation, alertes organisationnelles — prolongent cette orientation en reliant chaque action RH à un signal projet ou compétence préalablement identifié. Le manager n'envoie pas une demande générique : il formalise un `skill_gap` ou une `reallocation` en s'appuyant sur des analyses Matchmaker ou Watchdog, ce qui améliore la qualité du dialogue entre ligne managériale et fonction RH. Du point de vue des systèmes d'information, il s'agit d'un couplage faible mais sémantiquement riche entre modules, réalisé par des événements HTTP et des statuts de workflow plutôt que par une base unique monolithique.

2.3 Gestion des talents et des compétences

2.3.1 Talent Management

Le talent management [13, 2] désigne l'ensemble des pratiques visant à attirer, développer, mobiliser et fidéliser les collaborateurs en alignement avec la stratégie de l'organisation. La littérature managériale insiste sur la dimension anticipative : identifier les profils à potentiel, préparer les relais, adapter les parcours de développement et sécuriser la continuité des compétences critiques. Dans un contexte projet, le talent management ne se réduit pas aux processus annuels d'entretien ; il devient continu, alimenté par les affectations, les retours d'expérience et les signaux de surcharge ou de démotivation.

Strategic Copilot opérationnalise cette vision pour trois publics. L'espace manager (/workspace/manager) permet de piloter l'équipe via `TeamPage`, d'ouvrir une fiche talent détaillée (`TalentDetailPage`) et de lancer des analyses assistées (`use_ai: true` sur les scans Watchdog). L'espace RH centralise le référentiel employés (`RhEmployeesPage`), la mobilité (`RhMobilityPage`) et les plans de formation (`RhTrainingPlansPage`). L'espace talent offre une lecture individuelle des projets assignés, des tâches, de la charge et des compétences. Cette tripartition respecte les responsabilités respectives tout en s'appuyant sur un référentiel talent unique en base, évitant la duplication des profils.

Le document Agentic AI associe explicitement le **Talent Matching Agent** au workflow `WF_Talent_Matching` et à l'endpoint `/webhook/api/project/talents`. L'agent ne se contente pas de lister des noms : il évalue l'adéquation compétences–besoins, met en évidence les écarts et alimente les recommandations de staffing ou de montée en compétence affichées dans le tableau de bord manager et dans le modal Mission Control. Le talent management y gagne une dimension prescriptive, toujours soumise à validation humaine, conformément à l'éthique de l'aide à la décision.

2.3.2 Skill Management

Le skill management (gestion des compétences) structuré repose sur un référentiel de compétences (`skills`), des déclarations ou évaluations par collaborateur (`talent_skills`) et des besoins exprimés au niveau projet (`project_requirements`). Les approches matures distinguent compétences techniques, comportementales et certifications, avec des niveaux de maîtrise et des dates de validité. L'enjeu est de rapprocher l'offre de compétences de la demande induite par le portefeuille projet, afin de détecter précocement les ruptures.

Dans Strategic Copilot, le catalogue RH (`RhSkillsCatalogPage`) et la page des écarts critiques (`RhCriticalGapsPage`) matérialisent ce référentiel côté fonction RH. Côté manager, le panneau de matching (`talent-matching-panel`) et les blocs `matchmaker` du

tableau de bord restituent les sorties du workflow de matching sans recalcul front-end. Lorsqu'un écart persiste, le manager peut formaliser une demande RH de type `skill_gap` ou `training`, créant une chaîne fermée entre détection analytique et action RH traçable.

La littérature sur les compétences souligne également l'importance de la mise à jour continue des référentiels. Le processus P0 (`WF_Data_Sync`) décrit dans le cahier Agentic AI assure la synchronisation des données projets et talents, condition nécessaire à des analyses de matching non obsolètes. Strategic Copilot traite donc le skill management comme un flux vivant, synchronisé et exploité par des agents spécialisés, plutôt comme une photographie statique consultée une fois par an.

2.3.3 Workforce Planning

Le workforce planning (planification des effectifs et des capacités) vise à aligner les ressources disponibles sur la demande future, en intégrant contraintes légales, budgets et calendriers projet. Il s'agit d'arbitrer entre recrutement, formation, externalisation et réallocation interne. Les modèles classiques s'appuient sur des scénarios et des indicateurs de charge ; les approches avancées intègrent des simulations pour tester l'impact d'une hypothèse avant décision.

Strategic Copilot contribue à ce domaine par la visualisation des charges (`workload_snapshots`, pages équipe et talent), par les alertes de surcharge détectées par l'agent Risk & Weak Signal et par le simulateur what-if (`WhatIfProvider`, `POST /webhook/api/project/what-if`). Ce dernier permet de modifier paramétriquement une allocation, un délai ou une compétence cible (`allocation_pct`, `delay_days`, `training_skill_id`) et d'observer le delta sur le score de viabilité avant engagement. Le processus P5 (`WF_What_If`) correspond à cette logique de planification par scénarios, reconnue dans la littérature comme un prolongement naturel du workforce planning vers l'aide à la décision interactive.

Le workforce planning moderne exige également une lecture transverse RH-projet. En rapprochant l'espace RH (mobilité, formation) et l'espace manager (affectations `wmp-assign-v1`, portefeuille kanban), Strategic Copilot évite que la planification des capacités soit menée en vase clos, déconnectée des échéances et des risques projet.

Les travaux académiques sur la planification des effectifs soulignent par ailleurs l'importance des indicateurs prévisionnels : taux d'occupation, horizon de disponibilité, coût marginal d'une heure supplémentaire. Strategic Copilot ne prétend pas implémenter un optimiseur linéaire complet ; en revanche, les snapshots de charge et les alertes de surcharge fournissent des signaux exploitables par le décideur, complétés par le score de capacité intégré dans la viabilité (35 % du score global). L'approche est donc pragmatique : enrichir le jugement humain plutôt que substituer une planification centralisée opaque, ce qui rejoint les

critiques adressées aux systèmes expert trop rigides dans les années 1990 et justifie le recours contemporain à des architectures agentiques modulaires.

2.4 Gestion de portefeuille de projets

2.4.1 Project Portfolio Management

Le project portfolio management (PPM) [10, 4] désigne le pilotage d'un ensemble de projets visant à maximiser la valeur stratégique sous contraintes de ressources et de risques. Les cadres méthodologiques, notamment ceux diffusés par le Project Management Institute, insistent sur la priorisation, l'équilibre du portefeuille et la revue périodique de la performance collective. Le PPM diffère de la gestion opérationnelle d'un projet unique : il s'agit de choisir quels projets poursuivre, suspendre ou arrêter au regard d'indicateurs agrégés.

Strategic Copilot cible précisément ce niveau de pilotage pour les managers. `ProjectsPage` et `getCopilotProjects` offrent une vue portefeuille ; le tableau de bord Copilot (`getCopilotDashboard`) synthétise l'état global. Le composant `ProjectLifecycleStepper` matérialise les statuts `planned`, `active`, `on_hold`, `completed` et `cancelled`, facilitant la lecture du cycle de vie de chaque initiative. Le modal `ProjectMissionControlModal` concentre, pour un projet donné, les onglets `overview`, `équipe`, `tâches`, `risques`, `simulation` et `décisions`, incarnant la philosophie PPM d'une vision à 360 degrés avant arbitrage.

La décision stratégique `Continue`, `Adjust` ou `Stop`, produite par le Strategic Orchestrator, prolonge les grilles de priorisation PPM par une recommandation explicite fondée sur un score de viabilité composite. Le manager conserve la responsabilité de la validation, enregistrée dans le journal des décisions, ce qui respecte la distinction classique entre recommandation système et décision humaine.

2.4.2 Allocation des ressources

L'allocation des ressources consiste à affecter des talents à des projets ou des tâches en respectant disponibilité, compétences et équilibre de charge. La littérature opérationnelle modélise souvent ce problème comme une affectation sous contraintes multiples, parfois NP-difficile, d'où l'intérêt d'outils d'aide calculant des propositions plutôt que des solutions manuelles exhaustives.

Strategic Copilot implémente l'allocation via `project_assignments`, les webhooks `wmp-assign-v1` et `wmp-unassign-v1`, et les tâches projet (`wmt-list-v1`, `wmt-create-v1`). Le Talent Matching Agent propose des affectations compatibles ; le manager valide ou ajuste. Une option de recompute (variable d'environnement `VITE_TRIGGER_PROJECT_RECOMPUTE_AFTER_ASSIGNMENT`) permet de recalculer les propositions après une modification.

peut relancer la chaîne analytique après mutation, garantissant que les scores affichés reflètent la nouvelle configuration. Cette boucle mutation–recompute–affichage illustre une allocation dynamique, réactive aux changements terrain.

L'allocation ne se limite pas aux effectifs internes : les demandes RH de type `recruitment` ou `reallocation` traduisent des besoins que le portefeuille ne peut satisfaire en l'état. Strategic Copilot relie ainsi l'allocation tactique (qui fait quoi demain) à l'allocation stratégique (quels moyens RH engager), cohérent avec les modèles intégrés PPM–workforce planning.

2.4.3 Indicateurs de performance et de viabilité

Les indicateurs de performance projet (KPI) mesurent avancement, respect des délais, consommation budgétaire et qualité des livrables. La viabilité, concept mobilisé dans le document Agentic AI, désigne une capacité durable à atteindre les objectifs compte tenu des compétences disponibles, de la charge et des risques. Strategic Copilot combine ces deux dimensions : le **Project Analysis Agent** (`WF_Project_Analysis`) alimente les KPI et l'« atmosphère » projet, tandis que le Strategic Orchestrator agrège Skills Fit (40 %), Capacité (35 %) et Budget (25 %) en un score normalisé sur dix.

Les seuils de décision documentés (Continue si score $\geq 7,5$, Adjust entre 4 et 7,5, Stop en dessous de 4) fournissent une échelle de lecture commune, comparable aux tableaux de bord PPM utilisant des feux tricolores ou des zones de vigilance. Côté interface, `viability_score` et `risk_score` sur l'entité `Project` sont strictement en lecture seule ; le front-end affiche les valeurs renvoyées par `/webhook/api/project/viability` sans les recalculer.

Les graphiques Recharts sur `DecisionLogPage` et les visualisations du tableau de bord complètent les indicateurs scalaires par une représentation temporelle ou comparative, facilitant l'analyse des tendances. Cette dualité chiffre–graphique correspond aux bonnes pratiques des tableaux de bord décisionnels, où la synthèse numérique et la perception visuelle se renforcent mutuellement.

La gestion de portefeuille implique enfin une dimension temporelle : un projet peut être viable aujourd'hui et fragile dans trois mois si une compétence clé arrive à expiration ou si la charge cumulée dépasse un seuil. L'agent Risk & Weak Signal (`WF_Project_Risk`, `scan /webhook/api/watchdog/scan`) adresse cette dimension en détectant des signaux faibles avant dérive manifeste. Le manager peut déclencher un scan depuis `TeamPage` ou `RisksPage` avec `use_ai : true`, illustrant comment le PPM contemporain intègre la veille automatique plutôt que la seule revue mensuelle statique. Strategic Copilot ne remplace pas les comités de pilotage ; il leur fournit un pré-traitement analytique homogène, réduit le temps de collecte et oriente l'attention vers les projets et les talents présentant les scores les plus critiques.

2.5 Intelligence artificielle appliquée à l'aide à la décision

2.5.1 Systèmes d'aide à la décision

Les systèmes d'aide à la décision (SAD) [14, 21, 1], conceptualisés dès les travaux de Simon sur la rationalité limitée, visent à augmenter la capacité de traitement de l'information des décideurs sans se substituer à leur jugement. Un SAD typique combine une base de données, un module de modèles et une interface utilisateur, produisant des analyses, des simulations ou des recommandations structurées. Dans l'entreprise contemporaine, les SAD se sont enrichis de capacités prédictives et prescriptives, tout en conservant l'exigence d'explicabilité et de contrôle humain.

Strategic Copilot se définit explicitement comme une plateforme d'aide à la décision SaaS. Elle ne déclenche pas automatiquement des mutations critiques sans validation : le Strategist propose des options (`strategist/propose`) avant exécution (`strategist/execute`) ; le Helper suggère des actions (`suggested_actions`) que l'utilisateur peut accepter ou ignorer. Le Copilot global (`CopilotProvider`, `CopilotPanel`) restitue synthèse, risques, recommandations et niveau de confiance selon le contexte (`dashboard`, `project_detail`, `staffing`).

L'architecture multi-agents renforce la qualité de SAD en décomposant le problème : analyse projet, matching, risques, orchestration. Chaque agent joue le rôle d'un module spécialisé ; l'orchestrateur joue celui d'un agrégateur cohérent. Les traces affichées (`buildAgentTraces`) améliorent l'explicabilité en indiquant quel workflow a produit quel résultat et avec quelle fraîcheur (`fresh`, `aging`, `stale`), répondant aux critiques adressées aux « boîtes noires » décisionnelles.

2.5.2 Large Language Models

Les grands modèles de langage (LLM) [3, 12], dont la famille GPT développée par OpenAI [9] constitue la référence la plus diffuse, ont transformé l'interaction homme-machine dans les domaines textuels : synthèse, question-réponse, génération de recommandations argumentées. Leur intégration dans les systèmes d'entreprise soulève des questions de confidentialité, de coût, de latence et de fiabilité des sorties, qui imposent de les encapsuler derrière des orchestrateurs contrôlés plutôt que de les exposer directement aux clients lourds.

Dans Strategic Copilot, les LLM ne sont pas embarqués dans le bundle React. Les workflows n8n activent l'IA via le paramètre `use_ai` : `true` sur les appels risques, matching, viabilité ou Helper (`/webhook/api/helper/chat`). L'instance de production (`n8nprod.aphelionxinnovat`) héberge les graphes qui invoquent les API compatibles OpenAI, conformément au cahier des charges du PFE. Le front-end transmet le contexte métier (identifiant projet, page courante)

et affiche la réponse structurée, sans détenir les clés secrètes du modèle.

Le Helper Copilot illustre l’usage conversationnel du LLM dans un SAD : questions rapides métier, historique de conversations (`/webhook/manager/conversations`), validations asynchrones (`/webhook/api/helper/validations`). Cette conception respecte le principe de séparation des préoccupations : l’interface gère l’expérience ; n8n gère le prompt, l’appel modèle et la persistance éventuelle des échanges.

2.5.3 Recommandations automatisées

Les systèmes de recommandation, issus initialement du commerce en ligne, ont été adaptés aux contextes RH et projet : suggérer un profil pour une mission, une formation pour combler un écart, une action de mitigation pour un risque. Les approches hybrides combinent filtrage collaboratif, règles métier explicites et modèles d’apprentissage, avec un score de confiance associé à chaque proposition.

Strategic Copilot produit des recommandations à plusieurs niveaux. Le tableau de bord manager expose les blocs `matchmaker` et `analyst` lorsque le backend les fournit. Le Copilot global agrège risques et pistes d’action. Le matching talent retourne des profils classés et des écarts de compétences. Les demandes RH matérialisent les recommandations acceptées dans un workflow traçable. Le document Agentic AI insiste sur la transformation d’analyses multi-agents en décisions lisibles, sans automatisation aveugle : la recommandation reste une proposition jusqu’à validation explicite.

La qualité des recommandations dépend de la fraîcheur des données et de la chaîne P0–P5. Le recompute (`/webhook/api/copilot/recompute`) permet de régénérer les analyses après une mutation, limitant le risque de conseils obsolètes. Cette boucle de rafraîchissement s’inscrit dans la littérature sur les SAD adaptatifs, capables de se recalibrer lorsque l’état du système change.

Les débats récents sur l’IA en entreprise mettent en garde contre les hallucinations et les biais des LLM. Strategic Copilot atténue ces risques par plusieurs mécanismes convergents : ancrage des réponses dans des données PostgreSQL structurées, découpage du raisonnement en agents spécialisés aux périmètres plus étroits, affichage des traces de workflow et des niveaux de confiance lorsque le backend les fournit, et surtout absence d’exécution automatique des actions critiques sans validation via Strategist ou demandes RH. Cette posture « human-in-the-loop » n’est pas un repli technologique ; elle correspond à la doctrine des systèmes d’aide à la décision et aux attentes des encadrants métier exprimées dans le document Agentic AI.

2.5.4 Architectures multi-agents et Agentic AI

Au-delà des LLM monolithiques, la recherche et l'industrie s'orientent vers des architectures *agentiques* : plusieurs agents autonomes ou semi-autonomes collaborent sous la coordination d'un orchestrateur. Le paradigme Agentic AI, explicitement retenu pour Strategic Copilot, distingue l'agent Analyst (analyse projet), l'agent Matchmaker (matching), l'agent Watchdog (risques), l'agent Strategist (arbitrage) et l'orchestrateur stratégique qui agrège leurs sorties. Cette décomposition améliore la maintenabilité : faire évoluer le matching sans remettre en cause le calcul de risque, ou ajuster les pondérations de viabilité sans modifier l'interface Helper.

Le front-end matérialise cette architecture par `buildAgentTraces` et les libellés de workflow (`WF_Project_Analysis`, `WF_Project_Risk`, `WF_Project_Viability`) visibles dans Mission Control. Pour l'utilisateur, l'agentique se traduit par une transparence accrue : il comprend quelles analyses ont été exécutées, depuis quand, et s'il peut les relancer (`can_rerun`). Sur le plan théorique, ce modèle prolonge les systèmes multi-agents classiques en les couplant à des services de langage pour la formulation des recommandations, tout en conservant des scores numériques auditables stockés en base.

2.6 Automatisation des processus métier

2.6.1 Workflow Automation

L'automatisation des processus métier (BPA) vise à réduire les tâches manuelles répétitives, à accélérer les circuits de validation et à garantir l'application systématique des règles. Les plateformes BPMN historiques ont laissé place, en partie, à des approches plus légères fondées sur des déclencheurs événementiels, des scripts et des connecteurs API. Dans le domaine RH-projet, l'automatisation concerne typiquement les demandes de formation, les validations d'affectation, les notifications d'alerte et la génération de rapports.

Strategic Copilot automatise ces flux via n8n plutôt que via un moteur BPM embarqué dans le front-end. Les demandes RH suivent un cycle PATCH documenté dans `rh-actions.api.ts`. Les notifications manager agrègent alertes risque et actions RH (`manager-notifications-topbar`). Les rapports board-pack PDF et l'envoi courriel passent par `use-reports-n8n`. Chaque automatisation est observable, versionnée (`wmp-detail-v1`) et configurable par variables d'environnement, ce qui facilite la maintenance en environnement de production.

L'automatisation ne supprime pas le contrôle humain sur les décisions sensibles : elle structure l'information et accélère les enchaînements une fois la décision prise. Cette nuance

est essentielle pour l’acceptabilité sociale des outils agentiques en entreprise.

2.6.2 Orchestration avec n8n

n8n [8] est une plateforme d’automatisation open source fondée sur des workflows visuels composés de nœuds (Webhook, Cron, HTTP Request, Postgres, Function, IF, Merge, Execute Workflow, Respond to Webhook). Elle se positionne comme alternative accessible aux suites d’intégration propriétaires, avec l’avantage de pouvoir héberger les graphes on-premise ou sur un cloud dédié et d’y intégrer directement des appels à des services d’IA.

Strategic Copilot repose sur n8n comme backend métier unique visible depuis `copilot-ui`. Les webhooks `/webhook/login`, `/webhook/api/copilot/*`, `/webhook/api/project/*` et `/webhook/manager/*` constituent le contrat d’intégration. Le proxy Vite en développement et le fichier `vercel.json` en production relient le front-end statique à l’instance `n8nprod.aphelionxinnovations.com`. Les workflows `WF_Project_Analysis`, `WF_Talent_Matching`, `WF_Risk_KPI`, `WF_Strategic_Orchestrator` et `WF_What_If` matérialisent la chaîne agentique décrite dans le document Agentic AI.

L’orchestration n8n permet également la composition : un workflow peut en exécuter un autre, reproduisant la hiérarchie orchestrateur–agents spécialisés. Le fichier `n8n/route-validate-claims` témoigne de l’intégration fine entre sécurité JWT et routes manager, illustrant que l’automatisation inclut les contrôles d’accès, pas seulement les traitements analytiques.

Comparée aux suites BPM traditionnelles, n8n offre une courbe d’apprentissage plus accessible pour des équipes projet mixtes (développeurs, intégrateurs, experts métier). Les graphes restent exportables et versionnables ; les chemins `wmp-detail-v1` et assimilés permettent de faire coexister plusieurs versions d’un même flux pendant une migration. Pour Strategic Copilot, ce choix a facilité l’itération pendant le stage : ajouter un nœud Postgres, un appel OpenAI ou une branche conditionnelle sans redéployer l’intégralité du front-end. La contrepartie est une dépendance à la disponibilité de l’instance n8n, compensée côté client par des timeouts configurables (`VITE_HTTP_TIMEOUT_MS`) et des messages d’erreur explicites (`user-facing-api-error.ts`).

2.7 Technologies web modernes

2.7.1 React.js et Vite

React.js [6], bibliothèque JavaScript maintenue par Meta, repose sur une interface déclarative composée de composants réutilisables et d’un cycle de rendu optimisé par un DOM virtuel. Son écosystème (React Router, gestion d’état, hooks) en fait un standard pour les

applications web riches. Vite, outil de build moderne, accélère le développement grâce au rechargement à chaud et produit des bundles optimisés pour la production via Rollup.

Le projet Strategic Copilot utilise React 19 et Vite 7 [6, 22, 7] (`package.json` à la racine du dépôt), avec TypeScript pour le typage statique et le plugin `@vitejs/plugin-react-swc` pour la compilation rapide. `copilot-ui/src/main.tsx` structure l'application : providers (`AuthProvider`, `QueryClientProviderWrapper`, `CopilotProvider`, `WhatIfProvider`), routage React Router 7 et protection par rôle (`ProtectedRoute`). TanStack React Query 5 gère le cache des appels API, les invalidations après mutations et la résilience face à la latence des workflows n8n.

Ce choix technologique répond aux exigences d'une SPA multi-rôles évolutive : modularité des pages manager, RH et talent, réutilisation des composants Untitled UI et React Aria, séparation claire entre présentation et logique métier serveur.

L'architecture front-end adopte par ailleurs des patterns reconnus de qualité logicielle. Les hooks personnalisés (`useProjects`, `use-manager-risk-data`, `useRhActionsListQuery`) encapsulent les appels réseau et les règles d'invalidation du cache. Les formulaires critiques s'appuient sur `react-hook-form` et `zod` pour la validation côté client des saisies, sans confondre validation de saisie et calcul décisionnel. Les redirections legacy (`LegacyProjectDetailRedirect`, etc.) témoignent d'une évolution progressive des routes vers les workspaces `/workspace/manager`, `/workspace/rh` et `/workspace/talent`, pattern courant dans les applications SaaS matures qui doivent préserver les signets et liens profonds des utilisateurs.

2.7.2 Tailwind CSS

Tailwind CSS [18] adopte une approche utility-first : des classes atomiques composent l'interface sans imposer un catalogue de composants visuels rigide. La version 4, intégrée via `@tailwindcss/vite`, simplifie la configuration et améliore les performances de build. Couplé à `tailwind-merge` et aux plugins `typography` et `react-aria-components`, Tailwind assure la cohérence visuelle et l'accessibilité.

Strategic Copilot s'appuie sur Tailwind pour l'ensemble des écrans, du tableau de bord aux modales Mission Control, en mode clair et sombre (`ThemeProvider`, styles Recharts dans `globals.css`). L'internationalisation (i18next, locales FR/EN/AR dans `manager-workspace-locales.ts`) s'ajoute à cette couche présentation pour répondre à des utilisateurs multilingues, fréquents dans les contextes industriels internationaux.

2.7.3 Supabase et PostgreSQL

PostgreSQL [19, 15] est un système de gestion de bases relationnelles open source reconnu pour sa conformité SQL, ses transactions ACID et son extensibilité. Supabase propose une couche managée au-dessus de PostgreSQL : hébergement, API auto-générées, authentification et stockage objet. Dans les architectures modernes, Supabase est souvent mobilisé comme Backend-as-a-Service complémentaire à des orchestrateurs spécialisés.

Strategic Copilot utilise PostgreSQL comme socle des entités métier et des tables de résultats analytiques (`projects`, `talents`, `project_assignments`, `project_viability_scores`, etc.). Supabase intervient notamment pour le stockage fichier (avatars, PDF de rapports). Le front-end n'exécute pas de requêtes SQL directes pour les calculs décisionnels : il consomme les webhooks n8n qui encapsulent l'accès Postgres via des nœuds dédiés, renforçant la sécurité et la centralisation des règles métier.

Cette architecture hybride — PostgreSQL pour la persistance, n8n pour la logique, React pour l'expérience — illustre une tendance actuelle des applications SaaS métier : ne pas tout concentrer dans un unique framework full-stack, mais composer des services best-of-breed reliés par des API.

2.7.4 OpenAI et IA générative

OpenAI a popularisé l'IA générative grand public via les modèles GPT, accessibles par API et intégrables dans des chaînes de traitement. Les bonnes pratiques d'intégration recommandent de limiter l'exposition des clés API, de structurer les sorties (JSON) pour faciliter leur consommation par des applications typées, et de prévoir des garde-fous sur les entrées sensibles.

Strategic Copilot suit ces recommandations : les appels aux modèles compatibles OpenAI sont réalisés dans les workflows n8n lorsque `use_ai: true` est transmis depuis le front-end (`agentsApi.riskKpi`, `matching`, `Helper`, `viabilité`). Les types TypeScript (`api.types.ts`) définissent les enveloppes de réponse attendues ; les utilitaires (`flattenN8nRow`, `rh-api-parse.ts`) normalisent les variations de format renvoyées par les webhooks. Aucune clé OpenAI n'apparaît dans le code client, ce qui réduit la surface d'attaque et simplifie la rotation des secrets côté infrastructure.

L'IA générative intervient ainsi comme couche d'enrichissement linguistique et analytique au-dessus de règles et de requêtes SQL, et non comme substitut à la modélisation relationnelle des talents et des projets.

2.7.5 Visualisation de données avec Recharts

La visualisation de données facilite la détection de tendances, de ruptures et d'écarts par rapport à des seuils. Dans les tableaux de bord décisionnels, les graphiques en barres, lignes ou aires complètent les indicateurs chiffrés. Recharts [11, 20], bibliothèque React construite sur D3, propose des composants déclaratifs adaptés aux tableaux de bord métier.

Strategic Copilot intègre Recharts notamment sur `DecisionLogPage` et dans les visualisations du tableau de bord manager, avec des styles dédiés dans `globals.css` pour assurer la lisibilité en mode sombre. Le bundler Vite déduplique Recharts pour éviter les conflits de versions, détail technique mentionné dans la configuration du projet. Les graphiques restituent des séries temporelles ou comparatives issues des API, sans agrégation analytique supplémentaire côté client, respectant le principe de non-recalcul des scores dans `crud-domain.ts`.

La littérature sur la visualisation analytique rappelle que un graphique mal choisi peut induire en erreur ; Recharts, utilisé de manière ciblée sur des jeux de données déjà validés par le backend, limite ce risque. Les rapports PDF générés (`ReportsPage`, `board-pack`) complètent l'expérience écran par un livrable exportable pour les instances de gouvernance, prolongeant la fonction de reporting des SIRH et outils PPM classiques vers un format adapté aux revues de direction.

2.7.6 Sécurité, authentification et déploiement SaaS

Une plateforme SaaS d'aide à la décision doit intégrer dès la conception les exigences de sécurité : authentification, autorisation par rôle, protection des secrets et isolation des environnements. Strategic Copilot s'authentifie via `POST /webhook/login` et renouvelle les jetons (`/webhook/refresh`) ; `ProtectedRoute` empêche un talent d'accéder aux écrans manager. Les clés OpenAI et les credentials Postgres demeurent côté n8n et Supabase. Le déploiement du front-end en statique (build Vite, hébergement type Vercel avec relais `vercel.json`) sépare la surface d'exposition du moteur métier, ce qui réduit l'empreinte serveur côté présentation et facilite la montée en charge horizontale des workflows indépendamment de l'UI.

Cette architecture de sécurité en profondeur n'est pas accessoire : elle conditionne l'acceptation d'un copilot manipulant des données RH sensibles (compétences, charge, statuts contractuels implicites). En alignant les fondements de la cybersécurité applicative sur les choix d'implémentation observés, Strategic Copilot se place dans le cadre des bonnes pratiques des SaaS métier contemporains.

2.8 Synthèse et positionnement de Strategic Copilot

Le tableau 2.1 synthétise le croisement entre les domaines étudiés et les choix concrets du projet Strategic Copilot.

TABLE 2.1 – Positionnement de Strategic Copilot au regard des fondements étudiés

Domaine	Concept clé	Traduction dans Strategic Copilot
RH digitale	Vision consolidée	Trois espaces; données PostgreSQL unifiées.
Talent	Matching compétences	Agent WF_Talent_Matching; catalogue RH.
Workforce	Simulation capacités	What-if P5; snapshots de charge.
PPM	Pilotage portefeuille	Copilot dashboard; Mission Control.
Viabilité	Score et décision	Orchestrator; Continue/Adjust/Stop.
SAD	Aide, pas substitution	Validation humaine; journal décisions.
LLM	Synthèse et dialogue	Helper; use_ai côté n8n.
Automatisation	Workflows	n8n; demandes RH; notifications.
Web	SPA moderne	React 19, Vite 7, TypeScript, Tailwind 4.
Données	Source de vérité	PostgreSQL/Supabase; API-first.

Strategic Copilot se positionne comme une **plateforme d’aide à la décision agentique** pour l’écosystème Augmented Talents. Elle ne remplace ni le SIRH administratif ni l’outil de gestion de tâches classique : elle superpose une couche d’intelligence et d’orchestration reliant projets, talents, risques et recommandations. Son originalité tient à la combinaison d’une architecture multi-agents (Project Analysis, Talent Matching, Risk & Weak Signal, Strategic Orchestrator, Helper), d’une interface React typée consommant exclusivement des webhooks n8n, et d’une gouvernance explicite des décisions humaines.

Les fondements théoriques revus dans ce chapitre justifient les contraintes observées dans le code : pas de recalcul front des scores, séparation présentation–orchestration, traçabilité des demandes RH et des décisions, simulations avant arbitrage. Ils préparent le lecteur au chapitre suivant, qui formalise le cahier des charges et la planification, puis aux chapitres de réalisation et de tests qui décriront la mise en œuvre effective de ces principes dans le dépôt frontend-ui.

2.9 Conclusion

Ce chapitre a présenté les fondements sur lesquels repose Strategic Copilot : transformation digitale des RH, gestion des talents et des compétences, pilotage de portefeuille projet, systèmes d'aide à la décision enrichis par les grands modèles de langage, automatisation par workflows n8n et stack web moderne (React, Vite, Tailwind, PostgreSQL/Supabase, OpenAI orchestré côté serveur, Recharts). Chaque concept a été relié aux modules, agents et choix d'architecture effectivement implémentés dans la plateforme, sans extrapoler au-delà du périmètre vérifiable du code et du document Agentic AI.

Cette étude bibliographique éclaire la cohérence du projet : Strategic Copilot répond à un besoin réel de consolidation décisionnelle à l'intersection RH et projet, en s'appuyant sur des technologies et des modèles éprouvés, tout en respectant une exigence forte de contrôle humain et de traçabilité. Le chapitre suivant traduira ces fondements en spécifications fonctionnelles et non fonctionnelles détaillées, accompagnées d'une planification explicite du travail de fin d'études.

Chapitre 3

Cahier des charges et planification

3.1 Introduction

Le présent chapitre formalise le cahier des charges et la planification du projet Strategic Copilot. Il fait suite aux chapitres consacrés au contexte général, à l'analyse des besoins et à l'étude bibliographique : les acteurs, les tensions métier, les fondements théoriques et technologiques y ont été posés ; il s'agit ici de les traduire en spécifications fonctionnelles et non fonctionnelles, puis d'en décrire la mise en œuvre planifiée dans le temps. Le document de référence *PFE Agentic AI Strategic Talents* et le dépôt logiciel `frontend-ui` (application `copilot-ui`) constituent les deux sources de vérité retenues pour cette formalisation.

L'objectif est de présenter, de manière structurée et vérifiable, ce que le système doit faire, sous quelles contraintes, avec quelles ressources et selon quel calendrier. Le lecteur y trouvera la description du système multi-agents, le détail des fonctionnalités par espace utilisateur, les exigences de qualité, le découpage en phases, un diagramme de Gantt, la gestion des risques et l'inventaire des ressources mobilisées pendant le stage.

3.2 Cahier des charges

3.2.1 Présentation générale du système

Strategic Copilot est une plateforme SaaS intelligente d'aide à la décision pour la gestion stratégique des talents et des projets. Elle s'inscrit dans l'écosystème **Augmented Talents** sous le nom *Talent & Project Strategic Copilot*. Le système adopte une architecture **API-first** : l'interface React ne réalise aucun calcul décisionnel critique (scores de viabilité, risques, recommandations), conformément au principe explicite de `crud-domain.ts`.

Le cœur métier repose sur un **système multi-agents** orchestré par `n8n`. Quatre agents

principaux sont documentés dans le PFE Agentic AI : le **Project Analysis Agent** (analyse KPI et dérivés projet), le **Talent Matching Agent** (adéquation compétences et staffing), le **Risk & Weak Signal Agent** (risques et signaux faibles), et le **Strategic Orchestrator** (agrégation, score de viabilité, décision Continue/Adjust/Stop). Un **Helper Copilot** complète le dispositif par un dialogue contextualisé (WF_Copilot_Project_Detail, /webhook/api/helper/chat).

Les données sont persistées dans **PostgreSQL** via **Supabase** ; les workflows n8n lisent et écrivent les tables métier. Le front-end consomme les **webhooks n8n** (chemins /webhook/... et /api/... proxifiés). Les services d'**intelligence artificielle** (API compatibles OpenAI, activées par `use_ai: true` dans les workflows) sont invoqués exclusivement côté orchestrateur, jamais directement depuis le navigateur.

Le score de viabilité, défini dans le cahier Agentic AI, combine Skills Fit (40 %), Capacité (35 %) et Budget (25 %), chaque terme étant normalisé sur 10. Les seuils de décision sont : Continue si le score est supérieur ou égal à 7,5 ; Adjust entre 4 et 7,5 ; Stop en dessous de 4.

3.2.2 Acteurs du système

Le système distingue trois rôles applicatifs, matérialisés par le type `UserRole` (`manager`, `rh`, `talent`) et le routage `ProtectedRoute`.

Le **manager** pilote le portefeuille projet, supervise l'équipe, traite les alertes, formule des demandes RH, simule des scénarios what-if et enregistre ses décisions dans le journal Copilot. Il accède à l'espace `/workspace/manager`.

Le **responsable RH** administre le référentiel talents et compétences, suit les écarts critiques, les plans de formation, la mobilité, les alertes organisationnelles et traite les demandes des managers. Il accède à `/workspace/rh`.

Le **talent** consulte ses missions, projets, tâches, charge de travail, compétences et formations. Il accède à `/workspace/talent`.

En arrière-plan, les **workflows n8n** et la **base PostgreSQL** constituent des acteurs techniques : ils exécutent les règles, persistent les résultats analytiques dans `project_viability_scores` et `ai_decisions_log`.

3.2.3 Fonctionnalités principales

Les fonctionnalités principales, alignées sur le document Agentic AI et vérifiées dans le code, couvrent les domaines suivants.

La **visualisation** regroupe les tableaux de bord manager et RH, le portefeuille projets

(getCopilotDashboard, getCopilotProjects), les fiches détaillées et le modal Mission Control (ProjectMissionControlModal).

La **détection des risques** s'appuie sur l'agent Watchdog (WF_Project_Risk, WF_Risk_KPI), le scan manuel (POST /webhook/api/watchdog/scan), la page RisksPage et les notifications.

Les **recommandations IA** sont produites par le Matchmaker, l'Analyst, le Strategist et le Copilot global, avec affichage des niveaux de confiance et actions suggérées dans le Helper.

Les **demandes RH** typent les escalades (skill_gap, reallocation, training, overload, recruitment) et suivent un cycle de traitement par PATCH.

Les **simulations what-if** (POST /webhook/api/project/what-if, processus P5) permettent de tester l'impact d'une modification d'allocation, de délai ou de compétence avant arbitrage.

Les **rapports** (board-pack PDF, historique, envoi courriel via use-reports-n8n) supportent la gouvernance.

Le document Agentic AI structure ces capacités en six processus enchaînés, que le front-end déclenche ou consomme sans les réimplémenter. Le processus P0 (WF_Data_Sync) assure la synchronisation des référentiels projets et talents, condition préalable à toute analyse cohérente. Le processus P1 (WF_Project_Analysis) alimente les KPI et l'atmosphère projet affichés par l'Analyst Senior. Le processus P2 (WF_Talent_Matching) produit les propositions de staffing et les écarts de compétences exploités par le Matchmaker. Le processus P3 (WF_Risk_KPI et WF_Project_Risk) calcule les scores de fragilité et les alertes du Watchdog. Le processus P4 (WF_Strategic_Orchestrator) agrège les sorties précédentes en un score de viabilité et une décision Continue, Adjust ou Stop. Le processus P5 (WF_What_If) permet de simuler une modification d'allocation, de délai ou de compétence avant validation humaine. Cette chaîne constitue le fil conducteur du cahier des charges : chaque écran manager ou RH s'appuie sur une ou plusieurs étapes de ce pipeline.

3.2.4 Contraintes techniques et organisationnelles

Sur le plan **technique**, le front-end ne recalcule pas les scores analytiques ; toute évolution métier passe par n8n ; PostgreSQL est la source de vérité ; les secrets IA restent côté serveur ; les URLs sont configurables par variables VITE_* ; le proxy Vite assure la cohérence dev/prod avec n8nprod.aphelionxinnovations.com.

Sur le plan **organisationnel**, le projet s'inscrit dans une équipe répartissant les agents (analyse projet, matching, risques), sous encadrement EMSI (Mme Soundouss Abroun) et encadrants entreprise (MM. Youssef Moufak, Hamza Sadouk, Hicham Taoufik, Mohyi Eddine

Saadani). Les livraisons sont incrémentales, avec validation régulière des parcours critiques.

Le tableau 3.1 recense les principales interfaces HTTP consommées par `copilot-ui`. Ces contrats matérialisent le principe API-first : le navigateur n'accède jamais directement aux tables PostgreSQL pour les calculs analytiques.

TABLE 3.1 – Principales API Strategic Copilot (via n8n)

Méthode	Chemin (défaut)	Rôle
POST	/webhook/login	Authentification JWT.
GET	/webhook/api/copilot/dashboard	Vue portefeuille manager.
GET	/webhook/api/copilot/projects	Liste projets Copilot.
POST	/webhook/api/project/viability	Score et décision (Orchestrator).
POST	/webhook/api/project/what-if	Simulation de scénario.
POST	/webhook/api/project/risks	Risques Watchdog.
POST	/webhook/api/project/talents	Matching talents.
POST	/webhook/api/project/details	Analyse projet (Analyst).
POST	/webhook/api/strategist/propose	Options d'arbitrage.
POST	/webhook/api/strategist/execute	Exécution d'arbitrage.
POST	/webhook/api/helper/chat	Dialogue Helper Copilot.
POST	/webhook/api/copilot/recompute	Relance analyses projet.
GET	/webhook/manager/rh-actions	Liste demandes RH.
PATCH	/webhook/manager/rh-actions/:id	Traitement demande RH.
GET	/api/rh/dashboard	Tableau de bord RH.

Les chemins effectifs peuvent être surchargés par des variables d'environnement `VITE_*`, ce qui permet d'adapter l'instance n8n sans recompiler l'application lorsque les workflows sont versionnés (`wmp-detail-v1`, `wmt-list-v1`, `wmc-archive-v1`, etc.).

3.3 Spécifications fonctionnelles

3.3.1 Fonctionnalités de l'espace Manager

L'espace manager doit permettre l'authentification et l'accès au tableau de bord consolidé (`DashboardPage`), incluant les blocs `matchmaker` et `analyst` lorsque le backend les fournit.

Le manager doit pouvoir lister et ouvrir les projets (`ProjectsPage`, paramètre `openProjectId`), consulter le détail via `wmp-detail-v1` et piloter un projet dans le modal Mission Control : vue d'ensemble, équipe, tâches (`wmt-*v1`), risques, simulation, décisions.

Il doit gérer son équipe (`TeamPage`), consulter une fiche talent (`TalentDetailPage`), lancer un scan Watchdog individuel ou global, affecter ou désaffecter des talents (`wmp-assign-v1`, `wmp-unassign-v1`) avec `recompute` optionnel.

Il doit visualiser et traiter les risques (`RisksPage`), les notifications (`NotificationsPage`),

créer et suivre des demandes RH et talent, générer des rapports (`ReportsPage`, option `includeAi`), consulter le journal des décisions (`DecisionLogPage`) avec graphiques Recharts, utiliser le Helper (`HelperChatPage`) et le Copilot contextuel (`CopilotProvider`).

Le portefeuille projets doit permettre une lecture kanban ou tabulaire (`manager-projects-kanban`, `manager-projects-portfolio-table`) et un filtrage cohérent avec les statuts du cycle de vie (`planned`, `active`, `on_hold`, `completed`, `cancelled`) matérialisés par le composant `ProjectLifecycleStepper`. Lors d'une affectation, le système peut déclencher un recompute automatique si la variable `VITE_TRIGGER_PROJECT_RECOMPUTE_AFTER_ASSIGN` est activée, afin de rafraîchir les scores après mutation. Les alertes de risque doivent pouvoir être résolues ou ignorées via `PATCH /webhook/manager/risk-alerts/:id`, garantissant une boucle de traitement explicite plutôt qu'une simple consultation passive.

3.3.2 Fonctionnalités de l'espace RH

L'espace RH doit fournir un tableau de bord (`RhDashboardPage`, `/api/rh/dashboard`), la gestion des employés et talents (`RhEmployeesPage`), le catalogue de compétences (`RhSkillsCatalogPage`), l'identification des écarts critiques (`RhCriticalGapsPage`), les plans de formation (`RhTrainingPlansPage`), la mobilité et réallocation (`RhMobilityPage`), les alertes organisationnelles (`RhOrgAlertsPage`).

Il doit permettre le traitement des demandes managers (`rh/manager-requests`, `rh-actions.api.ts`), la gestion des comptes et sessions, les rapports, ainsi qu'une vue projets cohérente avec le portefeuille manager (`ProjectsPage`, `ProjectDetailsPage`).

Les demandes RH doivent respecter un cycle de vie traçable : création côté manager à partir d'une recommandation (écart de compétence, surcharge, besoin de formation ou de recrutement), notification, puis acceptation, rejet, avancement ou clôture côté RH via les mutations `PATCH` documentées dans `rh-actions.api.ts`. Le référentiel compétences (`Skill`, `TalentSkill`) doit rester aligné sur les besoins exprimés dans `project_requirements`, afin que les écarts critiques identifiés par le Matchmaker trouvent une réponse opérationnelle dans les plans de formation ou de mobilité.

3.3.3 Fonctionnalités de l'espace Talent

L'espace talent doit offrir un tableau de bord personnel (`TalentDashboardPage`), la consultation des projets assignés (`TalentProjectsPage`), des tâches (`TalentTasksPage`), de la charge (`TalentWorkloadPage`), des compétences (`TalentSkillsPage`) et des formations (`TalentTrainingPage`), ainsi que les notifications et le profil.

Les données affichées proviennent des API workspace talent (`/api/workspace/talent/*`) et respectent le périmètre du collaborateur connecté.

3.3.4 Fonctionnalités du Copilot intelligent

Le Copilot intelligent regroupe les capacités transverses d'aide à la décision. Le `CopilotProvider` expose des insights selon le scope (`dashboard`, `projects_list`, `project_detail`, `staffing`) via `getCopilotInsights` ou routes `/webhook/api/copilot/*`.

Le système doit afficher synthèse, risques, recommandations, score de viabilité et niveau de confiance, et permettre l'enregistrement d'une décision (`saveCopilotDecision`, `postStrategicDecision`).

Le Helper chat doit gérer les conversations (`/webhook/manager/conversations`), les messages (`helper/chat`), les questions rapides métier et les `suggested_actions` de navigation.

Le Strategist doit proposer des options d'arbitrage (`strategist/propose`) et les exécuter après validation (`strategist/execute`).

Le `recompute` (`/webhook/api/copilot/recompute`) doit relancer la chaîne d'agents après une mutation projet.

Les traces d'agents (`buildAgentTraces`) doivent indiquer le workflow associé, la fraîcheur du dernier calcul et la possibilité de relance.

Le simulateur what-if, porté par `WhatIfProvider`, doit accepter un identifiant de projet et des modifications paramétriques (`allocation_pct`, `added_talent_id`, `training_skill_id`, `delay_days`) et restituer un delta de score de viabilité, une décision proposée, un détail des contributions par agent et des recommandations associées dans `WhatIfResultPanel`. Le Strategist doit formaliser des options d'arbitrage (`reallocation`, `delay`, `reinforce`, `stop_scope`) avant toute exécution, conformément au principe de validation humaine inscrit dans le document Agentic AI. Enfin, les validations asynchrones déclenchées via `POST /webhook/api/helper/validations` complètent le dispositif lorsque le Helper propose des actions métier nécessitant une confirmation explicite.

3.4 Spécifications non fonctionnelles

3.4.1 Performance

L'application doit rester utilisable malgré la latence des workflows n8n. Les timeouts HTTP sont configurables (`VITE_HTTP_TIMEOUT_MS`, délai portefeuille `VITE_PORTFOLIO_TIMEOUT_MS`). React Query met en cache les réponses et invalide sélectivement après mutations (`invalidateManagerRisk` invalidation post-Strategist). Les listes volumineuses s'appuient sur la pagination ou les limites API (`limit` sur `getCopilotProjects`).

3.4.2 Sécurité

L'accès est contrôlé par JWT (`/webhook/login`, `/webhook/refresh`). Les routes sont protégées par rôle. Les clés API IA et secrets Supabase ne sont pas exposés dans le bundle client. Le fichier `n8n/route-validate-claims-merger-FIX.js` illustre la validation des claims JWT sur les routes manager sensibles.

3.4.3 Fiabilité

Le client HTTP tolère les réponses non JSON des webhooks (`http-client.ts`, `apiClient.ts`). Les erreurs sont traduites en messages utilisateur (`user-facing-api-error.ts`). Les conversations Helper gèrent les erreurs 400/401/403 avec retours explicites. Un échec de workflow n'entraîne pas de corruption d'état côté UI grâce à l'absence de recalcul local.

3.4.4 Maintenabilité

TypeScript structure les contrats (`api.types.ts`, `copilot.ts`, `crud-domain.ts`). Les hooks encapsulent les appels (`useProjects`, `use-manager-risk-data`, `useRhActionsListQuery`). Les URLs n8n sont centralisées (`build-n8n-url.ts`, `manager-projects-api.config.ts`). Le module `copilot/api/` parse et mappe les DTO de manière testable.

3.4.5 Scalabilité

L'architecture découplée permet de scaler horizontalement l'instance n8n et la base PostgreSQL indépendamment du front-end statique (`build Vite`, déploiement Vercel avec relais `vercel.json`). Les workflows versionnés (`wmp-detail-v1`, etc.) facilitent l'évolution sans rupture. L'ajout d'un nouvel agent se traduit par un nouveau workflow et des champs DTO, sans refonte globale de l'UI.

3.4.6 Ergonomie et expérience utilisateur

L'interface adopte Tailwind CSS 4, Untitled UI et React Aria pour la cohérence visuelle et l'accessibilité. L'internationalisation (`i18next`, locales FR/EN/AR dans `manager-workspace-locales.ts`) adapte les libellés. Le mode sombre et les styles Recharts (`globals.css`) assurent la lisibilité des graphiques. La navigation par rôle évite la surcharge cognitive : chaque espace expose uniquement les écrans pertinents.

3.5 Planification du projet

3.5.1 Méthodologie adoptée

La méthodologie suit une approche incrémentale inspirée des pratiques agiles, adaptée au contexte PFE. Chaque itération comprend une sélection de user stories, le développement front-end, l'intégration des webhooks n8n correspondants, des tests manuels sur les parcours critiques et une revue avec les encadrants. Les processus P0 à P5 du document Agentic AI (sync, analyse, matching, risques, orchestration, what-if) ont servi de colonne vertébrale pour prioriser les intégrations backend.

3.5.2 Découpage en phases

La phase 1 **Analyse et cadrage** a permis de stabiliser le cahier des charges Agentic AI, les personas et les contrats API cibles. La phase 2 **Conception** a défini l'architecture multi-agents, le modèle de données et la structure des trois espaces. La phase 3 **Réalisation** a livré les écrans manager, RH et talent, les intégrations CRUD et les workflows prioritaires. La phase 4 **Intégration IA** a connecté Copilot, Helper, Matchmaker, Watchdog, Strategist et Orchestrator. La phase 5 **Validation et rédaction** a couvert les tests, les corrections, les rapports PDF et le mémoire PFE.

Le tableau 3.2 synthétise, pour chaque phase, les objectifs principaux et les jalons de validation attendus avec les encadrants.

TABLE 3.2 – Phases du projet et jalons de validation		
Phase	Objectif	Jalon de validation
1 — Analyse	Cadrage Agentic AI, personas, contrats API	Cahier des charges validé.
2 — Conception	Architecture multi-agents, modèle de données	Schéma des workflows et écrans cibles.
3 — Réalisation	Espaces manager, RH, talent ; CRUD n8n	Parcours critiques démontrables.
4 — Intégration IA	Copilot, Helper, Orchestrator, what-if	Scores et alertes cohérents en base.
5 — Validation	Tests, corrections, rapport PFE	Démonstration de soutenance.

3.5.3 Diagramme de Gantt

La figure 3.1 présente une planification indicative sur le semestre de stage (septembre 2025 à avril 2026). Les dates sont ajustables selon le calendrier académique réel.

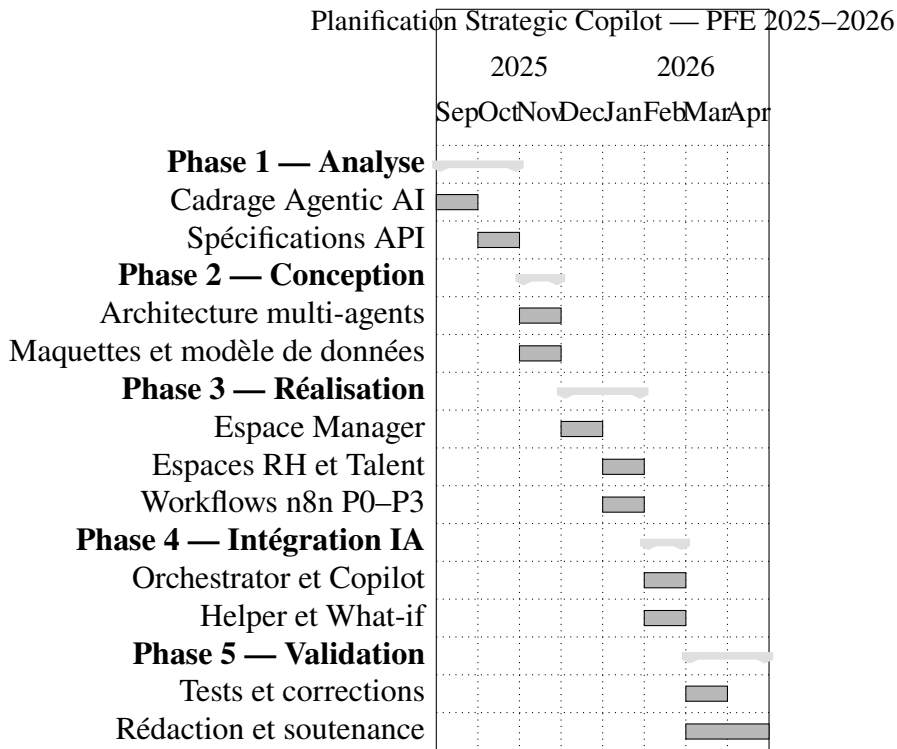


FIGURE 3.1 – Diagramme de Gantt du projet Strategic Copilot

3.5.4 Livrables du projet

Les livrables comprennent l’application web `copilot-ui` (build Vite), les workflows n8n documentés et versionnés, la configuration d’environnement (`.env`, variables `VITE_*`), le présent rapport LaTeX (`rapport/main.tex`), les rapports métier PDF (board-pack), les supports de soutenance prévus dans le document Agentic AI (grille jury, script dix minutes) et le dépôt Git `frontend-ui`.

Chaque livrable logiciel est assorti d’une preuve d’intégration observable dans l’interface : le build front-end démontre la couverture des trois rôles ; les workflows versionnés garantissent la reproductibilité des calculs ; le rapport formalise les choix et les écarts éventuels entre ambition documentaire et implémentation effective. Cette traçabilité répond à l’exigence académique du PFE tout en servant la maintenance industrielle du produit Augmented Talents.

3.6 Gestion des risques

3.6.1 Risques techniques

Les risques techniques identifiés incluent l'indisponibilité ou la lenteur de l'instance n8n, les ruptures de contrat JSON entre workflows et front-end, les erreurs 500 sur l'Orchestrator lors de données invalides (par exemple talent non UUID dans what-if, d'où la sanitization dans `orchestrator.api.ts`), et les dérives de configuration entre environnements (proxy Vite versus production Vercel).

3.6.2 Risques organisationnels

Les risques organisationnels concernent la disponibilité des encadrants pour valider les arbitrages métier, la qualité des données en base (impact direct sur la pertinence des alertes), et la dispersion des responsabilités entre plusieurs agents IA au sein de l'équipe projet.

3.6.3 Mesures d'atténuation

Les mesures d'atténuation retenues sont : timeouts et messages d'erreur explicites ; typage TypeScript strict ; tests répétés des parcours critiques ; documentation des webhooks ; revues régulières avec les encadrants ; principe « pas de calcul front » pour garantir une seule source de vérité analytique ; et sauvegarde du dépôt Git avec scripts de déploiement (`commit-et-push.cmd`).

3.7 Ressources utilisées

3.7.1 Ressources humaines

L'étudiant Ouissame Mahtour a assuré le développement front-end et l'agent Project Analysis. Un membre de l'équipe (Mohammed) a contribué à l'agent Talent Matching. L'agent Risk & Weak Signal a été co-construit. L'encadrement pédagogique a été assuré par Mme Soundouss Abroun (EMSI Tanger). L'encadrement entreprise a été assuré par MM. Youssef Moufak, Hamza Sadouk, Hicham Taoufik et Mohyi Eddine Saadani.

3.7.2 Ressources logicielles

Les ressources logicielles comprennent React 19, Vite 7, TypeScript, Tailwind CSS 4, React Router 7, TanStack React Query, axios, i18next, Recharts, n8n (instance `n8nprod.aphelionxinnovat`), PostgreSQL/Supabase, Cursor/VS Code, LaTeX Workshop (MiKTeX), Git, et les services d'IA configurés dans les workflows n8n (API compatibles OpenAI, paramètre `use_ai`).

3.7.3 Ressources matérielles

Les ressources matérielles comprennent les postes de développement des membres de l'équipe, l'accès réseau à l'instance n8n et à Supabase, et l'environnement de déploiement (Vercel pour le front-end statique). Aucune infrastructure propriétaire supplémentaire n'a été requise au-delà de ces services.

3.8 Conclusion

Ce chapitre a formalisé le cahier des charges et la planification de Strategic Copilot. Le système a été présenté comme une plateforme SaaS multi-agents, API-first, reposant sur n8n, PostgreSQL/Supabase et une interface React typée. Les spécifications fonctionnelles ont été déclinées par espace (manager, RH, talent) et pour le Copilot intelligent; les spécifications non fonctionnelles ont encadré performance, sécurité, fiabilité, maintenabilité, scalabilité et ergonomie. La planification par phases, le diagramme de Gantt, la gestion des risques et l'inventaire des ressources complètent ce cadrage.

Le chapitre suivant détaillera la conception fonctionnelle et technique complète telle qu'implémentée dans le dépôt `frontend-ui`.

Chapitre 4

Conception

4.1 Introduction

Le présent chapitre expose la conception fonctionnelle et technique de Strategic Copilot telle qu'elle résulte de l'implémentation réelle du dépôt `frontend-ui`. Il fait suite à l'étude bibliographique et au cahier des charges : les fondements théoriques et les exigences y ont été posés ; il s'agit ici de décrire comment ils se matérialisent dans l'architecture, les workflows n8n consommés par l'interface, le modèle de données PostgreSQL, la couche React et les mécanismes de sécurité. L'analyse s'appuie exclusivement sur le code source de `copilot-ui`, sur les fichiers de configuration (`vite.config.ts`, `vercel.json`, `.env.production`) et sur le script d'intégration `n8n/route-validate-claims-merger-FIX.js`. Les graphes n8n complets ne sont pas versionnés dans ce dépôt : les workflows sont identifiés par les noms explicites (`WF_*`) et par les chemins de webhooks versionnés (`wmp-*-v1`, `wmt-*-v1`) référencés dans le front-end.

L'objectif est de fournir une lecture architecturale exploitable en soutenance : comprendre qui appelle quoi, où sont calculés les scores, comment les trois espaces utilisateurs partagent les mêmes services, et comment l'IA (paramètre `use_ai : true`, modèles compatibles OpenAI côté n8n) reste encapsulée hors du navigateur. Le chapitre suivant du rapport pourra détailler la réalisation opérationnelle et les campagnes de tests ; celui-ci se concentre sur la conception telle qu'elle est observable dans le produit livré.

4.2 Architecture globale du système

4.2.1 Vue d'ensemble

Strategic Copilot est une application web SaaS organisée autour d'un client React (`copilot-ui`) et d'un backend métier constitué de workflows n8n exposés en HTTP. L'instance de production référencée dans le projet est `n8nprod.aphelionxinnovations.com`. Le front-end ne contient aucune logique de calcul des scores de viabilité, de risque ou de matching : le module `crud-domain.ts` impose que les champs analytiques soient en lecture seule et que toute recommandation provienne de l'API. PostgreSQL, accessible via Supabase pour l'hébergement et le stockage objet (PDF de rapports, fichiers), centralise les entités métier; les workflows n8n lisent et écrivent ces tables via des nœuds Postgres.

Trois espaces utilisateurs structurent l'expérience, routés dans `main.tsx` sous `/workspace/manager`, `/workspace/rh` et `/workspace/talent`. Un panneau Copilot global (`CopilotPanel`, `CopilotProvider`) et un contexte de simulation (`WhatIfProvider`) traversent l'application manager. L'authentification repose sur des jetons JWT obtenus par `/webhook/login` et renouvelés par `/webhook/refresh`, transportés par `httpClient` (axios) sur chaque requête métier.

4.2.2 Architecture multi-couches

La figure 4.1 résume les couches et leurs responsabilités.

TABLE 4.1 – Couches architecturales de Strategic Copilot

Couche	Rôle et artefacts
Présentation	React 19, Vite 7, TypeScript, Tailwind CSS 4; pages par workspace; composants Mission Control, Copilot, Helper.
Orchestration client	React Router 7, TanStack React Query 5, providers (Auth, Copilot, WhatIf, Toast).
Accès API	<code>httpClient</code> , <code>apiGet/apiPost</code> ; résolution d'URL (<code>build-n8n-url.ts</code> , <code>VITE_*</code>).
Intégration	Proxy Vite (dev) et <code>vercel.json</code> (prod) vers n8n; réécriture <code>/api</code> → <code>/webhook/api</code> .
Métier	Workflows n8n (webhooks versionnés et routes <code>/webhook/api/*</code>).
Données	PostgreSQL; Supabase (stockage fichier); pas d'accès SQL direct depuis React.
IA	Appels modèle dans n8n si <code>use_ai: true</code> ; API compatibles OpenAI.

En développement, le navigateur appelle `localhost:5173/webhook/...` ou `/api/...`; le proxy Vite relaie vers n8n avec `changeOrigin: true` et des délais portés à 60 secondes pour les mutations lourdes. En production sur Vercel, la règle `/webhook/:path*` relaie vers la même origine n8n, ce qui évite les problèmes CORS tout en gardant un build front-end statique.

4.2.3 Architecture multi-agents

Le système agentique n'est pas implémenté dans React : il est matérialisé par des workflows n8n distincts, agrégés conceptuellement par le Strategic Orchestrator (POST `/webhook/api/project/viability`) et par le recompute global (POST `/webhook/api/copilot/recompute`). Le fichier `agent-trace.ts` documente, pour l'affichage Mission Control, cinq traces : Analyst Senior (WF_Project_Analysis), Watchdog Senior (WF_Project_Risk), Strategist v2 (WF_Project_Viability), Talent Insights Senior (WF_Talent_Insights) et Helper Copilot (WF_Copilot_Project_Detail). Le matching projet-talents est associé dans le code au workflow WF_Talent_Matching (`project-talent-matching.api.ts`, `agents.api.ts`).

Chaque trace expose une fraîcheur (`fresh`, `aging`, `stale`) calculée côté client à partir des horodatages serveur (`analyzed_at`, `calculated_at`, `matched_at`), et un indicateur `can_rerun` pour autoriser une relance manuelle. Cette conception sépare clairement la visualisation du cycle de vie des analyses et l'exécution réelle des workflows côté n8n.

4.3 Conception fonctionnelle

4.3.1 Acteurs du système

Le type `UserRole` (`api.types.ts`) distingue `manager`, `rh` et `talent`. Le composant `ProtectedRoute` restreint les routes de chaque workspace. Le manager et le RH partagent certaines vues projet (`ProjectsPage`, `ProjectDetailsPage`) et la route `/workspace/rh/manager-requests` via `RhManagerRequestsEntry`, ce qui permet au RH de traiter les demandes initiées par les managers.

Les acteurs techniques comprennent les workflows n8n (déclencheurs Webhook, Cron, chaînes `Execute Workflow`) et PostgreSQL. Le document de cadrage Agentic AI mentionne des processus P0 à P5; seuls ceux dont les endpoints ou noms `WF_*` sont référencés dans `copilot-ui` sont retenus ici comme partie intégrante de la conception observable.

4.3.2 Cas d'usage

Les cas d'usage principaux se déclinent par espace sans recourir à une formalisation UML figée dans le dépôt, mais leur couverture est vérifiable par les routes de `main.tsx`.

Côté manager, un responsable se connecte, consulte le tableau de bord (`DashboardPage`, `/webhook/manager/dashboard` ou agrégats Copilot), parcourt le portefeuille (`ProjectsPageManager`, `getCopilotProjects`), ouvre le détail projet (`wmp-detail-v1`) et le modal Mission Control, affecte ou désaffecte des talents (`wmp-assign-v1`, `wmp-unassign-v1`), gère les tâches (`wmt-*-v1`), consulte les risques (`RisksPage`, `/webhook/api/project/risks`), lance un scan Watchdog (`/webhook/api/watchdog/scan`), simule un scénario what-if, crée ou traite des demandes RH, génère des rapports PDF, consulte le journal des décisions et dialogue avec le Helper (`HelperChatPage`).

Côté RH, un utilisateur administre le référentiel (`RhEmployeesPage`, catalogue `RhSkillsCatalogPage`), analyse les écarts (`RhCriticalGapsPage`), les plans de formation et la mobilité, traite les alertes organisationnelles, gère les comptes (`RhAccountsWorkspacePage`, `webhooks /webhook/rh/users`) et les sessions (`/webhook/rh/sessions`), et accède aux mêmes vues projet que le manager pour le pilotage transversal.

Côté talent, le collaborateur consulte son tableau de bord, ses projets, tâches, charge, compétences et formations via des endpoints dédiés `/api/workspace/talent/*`, et peut mettre à jour une tâche (`/api/talent/tasks/:id`) selon la configuration `VITE_TALENT_TASK_UPDATE_URL`.

4.3.3 Flux fonctionnels

Le flux décisionnel type commence par une consultation de projet, déclenche éventuellement une analyse (`/webhook/api/project/details`), un calcul de risques (`use_ai: true`), un matching (`/webhook/api/project/talents`), puis une viabilité (`/webhook/api/project/vi`) qui alimente la décision Continue, Adjust ou Stop. Le manager peut enregistrer sa décision (`/webhook/api/copilot/decision`, `/webhook/manager/copilot-decisions`) ou passer par le Strategist (`propose` puis `execute`). Une mutation d'affectation peut déclencher un recompute si `VITE_TRIGGER_PROJECT_RECOMPUTE_AFTER_ASSIGN=1`.

Le flux RH suit la création d'une demande (POST sur `/webhook/api/rh/actions` ou `/webhook/manager/rh-actions`), notification, puis traitement par PATCH `/api/rh/actions/:id` (proxy vers le webhook `UUID c8bae94d-...`) ou PATCH `/webhook/manager/rh-actions/:id`. Le flux Helper associe une conversation (GET/POST `/webhook/manager/conversations`) à des messages (`/webhook/api/helper/chat`) et permet l'archivage logique (PATCH `.../archive`, famille `wmc-archive-v1` configurable).

4.4 Conception de l'architecture agentique

4.4.1 Agents d'analyse

L'agent d'analyse projet correspond à `WF_Project_Analysis`, invoqué par `POST /webhook/api/project/analysis` avec un `project_id`. La réponse typée `ProjectAnalysisResponse` contient des KPI (`ProjectKpiFull`) : progression, santé projet, charge, retards et alertes structurées. L'interface restitue ces éléments dans Mission Control et dans les blocs `analyst` du tableau de bord lorsque le backend les fournit.

L'agent de risques (`WF_Project_Risk`, également lié aux KPI de fragilité) est appelé par `POST /webhook/api/project/risks` ou `GET` selon les écrans (`project-risks.api.ts`, `use-manager-risk-data`). Le corps inclut `use_ai: true` sur les pages `manager` (`RisksPage`, `TeamPage`). Les drivers (`anxiety_pulse`, `chronic_overload`, `skills_gap`, `talent_dependency`) structurent l'explication du score dans `workspace-manager.api.ts`.

4.4.2 Agents de recommandation

L'agent de matching `WF_Talent_Matching` produit la liste classée des talents compatibles (`TalentMatchingResponse`) via `POST /webhook/api/project/talents`. Le panneau `talent-matching-panel` affiche les résultats sans recalcul. `WF_Talent_Insights` complète la vue équipe projet dans les traces d'agents.

Le Helper (`WF_Copilot_Project_Detail`) fournit le dialogue contextualisé : `POST /webhook/api/helper/chat` avec `message`, `project_id` et `conversation_id`. Les `suggested_actions` orientent la navigation (assignation, formation, revue, risque). Les validations asynchrones passent par `POST /webhook/api/helper/validations`.

4.4.3 Agent orchestrateur

Le Strategic Orchestrator est matérialisé par `WF_Project_Viability` (trace `Strategist`) et par l'endpoint `POST /webhook/api/project/viability` (`orchestrator.api.ts`, `computeViability`). Le score composite pondère Skills Fit (40 %), Capacité (35 %) et Budget (25 %) selon le cahier Agentic AI. Le `recompute` `POST /webhook/api/copilot/recompute` relance la cascade après mutation.

Le Strategist métier (`POST /webhook/api/strategist/propose` et `execute`) propose des options d'arbitrage (`reallocation`, `delay`, `reinforce`, `stop_scope`). La réponse d'exécution peut déclencher `WF_Strategist_Accept` (réduction de périmètre, pause projet) avec des toasts métier définis dans `strategist-arbitrage.ts`.

4.4.4 Interaction entre agents

Les agents ne s'appellent pas depuis React : le chaînage est réalisé dans n8n par `Execute Workflow` et par le `recompute`. Le front-end enchaîne parfois manuellement plusieurs POST (par exemple viabilité puis affichage des traces) ou délègue tout au `recompute`. Le tableau 4.2 synthétise les interactions côté interface.

TABLE 4.2 – Agents, workflows et endpoints principaux

Agent	Workflow	Endpoint	Consommateur UI
Analyst	WF_Project_Analysis	POST .../project/details	Mission Control, dashboard
Watchdog	WF_Project_Risks	POST/GET .../project/risks	RisksPage, équipe
Matchmaker	WF_Talent_Matching	POST .../project/talents	Matching panel, Copilot
Talent Insights	In- WF_Talent_Insights	(clic sur détail projet)	Mission Control
Orchestrator	WF_Project_Viability	POST .../project/viability	Viabilité, Strategist
Helper	WF_Copilot_Project_Detail	POST .../helper/chat	Helper, Copilot panel
Strategist	(exécution)	POST .../strategist/*	Arbitrage projet

La simulation what-if (`POST /webhook/api/project/what-if`, processus P5 du cahier Agentic AI) consomme un `project_id` et des modifications (`allocation_pct`, `added_talent_id`, `training_skill_id`, `delay_days`) et alimente `WhatIfResultPanel` via `WhatIfProvider`.

4.5 Conception des workflows n8n

4.5.1 Principes d'orchestration

Les workflows n8n suivent un modèle `webhook-first` : un nœud `Webhook` reçoit la requête, des nœuds `Postgres` lisent ou écrivent les tables, des nœuds `Function` ou `HTTP` appellent l'IA si `use_ai` est vrai, et un nœud `Respond to Webhook` renvoie un JSON souvent encapsulé (`json, body`), aplati côté client par `flattenN8nRow` et des parseurs dédiés (`rh-api-parse.ts`, `copilot-api.parse.ts`).

Les chemins versionnés (`wmp-detail-v1`, `wmt-list-v1`, etc.) permettent d'évoluer les contrats sans casser immédiatement le front-end, grâce aux variables `VITE_WMP_*`,

VITE_WMT_* et VITE_MANAGER_PROJECTS_*. Le préfixe alternatif /n8n-webhook en développement réécrit vers /webhook pour les tâches projet (projectTasksApi.ts).

4.5.2 Description détaillée des workflows réellement présents

Le dépôt ne contient pas les exports JSON n8n; la table 4.3 recense les familles de workflows identifiées dans le code source, avec le chemin HTTP par défaut et le rôle.

Certaines routes utilisent un identifiant UUID de webhook n8n (d0f5b013-5af1-491f-ad4e-e463a6ac8bae94d-8de1-4f06-bb0a-a1e90eb6a80d/api/rh/actions) : ce pattern correspond à des workflows publiés avec un chemin stable côté instance n8n, indépendamment du nom affiché dans l'éditeur.

4.5.3 Communication entre workflows

La communication inter-workflows est interne à n8n (Execute Workflow, partage Postgres). Le front-end ne voit que des endpoints agrégés : par exemple le détail projet wmp-detail-v1 peut déjà embarquer analyse, risques, talents et viabilité, tandis que le recompute explicite relance la chaîne complète. Le proxy Vite traite différemment /api/rh/actions (rewrite vers le webhook UUID) et /api générique (préfixe /webhook), ce qui reflète deux stratégies de publication n8n coexistantes en production.

Le script route-validate-claims-merger-FIX.js fusionne la validation JWT avec le routage des requêtes manager team, preuve que la sécurité et l'orchestration sont couplées en amont des handlers métier.

4.5.4 Gestion des erreurs et reprise

Côté client, http-client.ts gère le refresh JWT sur 401, évite les boucles sur login et refresh, et émet des toasts globaux via HttpErrorToaster sauf si skipGlobalHttpErrorToast est défini (rapports, archivage conversations). user-facing-api-error.ts traduit les codes HTTP en messages métier, notamment pour Mission Control.

Les réponses n8n non JSON ou encapsulées sont tolérées ; les timeouts sont configurables (VITE_HTTP_TIMEOUT_MS, VITE_PORTFOLIO_TIMEOUT_MS). React Query réessaie jusqu'à deux fois les requêtes en échec transitoire (query-client-provider.tsx) et invalide les caches après mutations (risques, Strategist, portefeuille). En cas d'échec de l'Orchestrator, orchestrator.api.ts sanitise les identifiants talent invalides pour le what-if afin d'éviter les erreurs 500 liées à des UUID mal formés.

TABLE 4.3 – Workflows n8n et webhooks référencés dans copilot-ui

Famille / WF	Chemin ou préfixe HTTP	Rôle
Auth	/webhook/login, refresh, logout, /webhook/auth/me	JWT, profil, mot de passe
WF_Manager_Projects	/webhook/manager/projects, wmp-detail-v1, wmp-update-v1	Liste, détail, mise à jour projet
Affectations	wmp-assign-v1, wmp-unassign-v1	POST assign, DELETE unassign
WF_Manager_ProjectTasks	/api/tasks/create/update/ wmp-detail-v1/manager/team/ team-id	CRUD tâches projet
WF_Manager_Team	GET /webhook/manager/team	Famille talent, liste équipe
Watchdog	POST /webhook/api/watchdog/scan	Scan risques talent/projet
WF_Project_Analysis	POST /webhook/api/project/details	KPI projet
WF_Project_Risks	POST/GET .../api/project/risks	Risques, score fragilité
WF_Talent_Matching	POST .../api/project/talents	Matching compétences
WF_Project_Viability	POST .../api/project/viability	Score viabilité, décision
What-if	POST .../api/project/what-if	Simulation scénario
Copilot re-compute	POST .../api/copilot/recompute	Relance chaîne agents
Copilot vues	/webhook/api/copilot/dashboard/ projects, staffing, decision	Tableau de bord décisionnel
Strategist	.../strategist/propose, execute	Arbitrage
Helper	.../helper/chat, validations	Chat, validations
WF_Manager_RH	/webhook/manager/rh-actions, /webhook/api/rh/actions	Actions RH
RH dédié	PATCH /webhook/c8bae94d-.../api/rh/ actions/ demande	Mise à jour RH, demande
Alertes risque	PATCH /webhook/manager/risk-alerts/:id	Acquittement alerte
Décisions	GET /webhook/manager/copilot-decisions	Journal décisions
Conversations	GET/POST /webhook/manager/conversations, archive	Helper historique
wmc-archive-v1	PATCH .../conversations/:id/archive	Archivage logique
Dashboard manager	GET /webhook/manager/dashboard	KPI manager
RH agrégats	/api/rh/dashboard, critical-gaps, etc.	Espaces RH

4.6 Conception de la base de données

4.6.1 Architecture Supabase/PostgreSQL

PostgreSQL stocke les entités relationnelles et les tables de résultats analytiques. Supabase fournit l'hébergement managé et le stockage objet pour les livrables PDF (`file_url` dans `GenerateReportResponse`). Le front-end n'utilise pas le client Supabase pour les requêtes métier analytiques : les variables `VITE_SUPABASE_URL` et `VITE_SUPABASE_ANON_KEY` sont présentes pour d'éventuels usages fichier ou futurs, mais le chemin nominal passe par `n8n`.

4.6.2 Tables principales

Le cahier Agentic AI et les types TypeScript convergent vers le modèle suivant. La table `projects` porte les initiatives (statut, scores `viability_score`, `risk_score`, `decision` en lecture analytique). `project_tasks` et `project_requirements` décrivent le travail et les besoins. `project_assignments` lie talents et projets. `talents`, `talent_skills` et `skills` modélisent les personnes et le référentiel de compétences. `workload_snapshots` historise la charge. `project_viability_scores` et `ai_decisions_log` persistent les sorties de l'orchestrateur et la traçabilité des décisions IA et humaines.

Les objets de workflow (demandes RH, alertes, conversations Helper) possèdent des tables ou vues associées côté backend, manipulées uniquement par `n8n` ; le front-end les voit sous forme de DTO (`RhActionItem`, `ConversationItem`, alertes dans `notifications.api.ts`).

4.6.3 Relations entre entités

Un projet agrège des tâches et des exigences ; des affectations relient des talents au projet ; chaque talent possède des `talent_skills` pointant vers `skills`. Les snapshots de charge et les scores de viabilité sont dérivés : produits par les workflows, consommés en lecture seule par React. Les demandes RH référencent optionnellement un `project_id` et typent la demande (`skill_gap`, `reallocation`, `training`, `overload`, `recruitment`).

Le tableau 4.4 résume les relations fonctionnelles.

TABLE 4.4 – Entités relationnelles et dérivées

Entité	Liée à	Produite par
project_assignments	projects, talents	CRUD manager, n8n
project_viability_scores	projects	Orchestrator
ai_decisions_log	projects	Copilot, Strategist
workload_snapshots	talents	Sync / calcul n8n
talent_skills	talents, skills	RH, talent

4.7 Conception du frontend

4.7.1 Architecture React/Vite

L'application est une SPA montée par createRoot dans main.tsx. Vite 7 compile avec @vitejs/plugin-react-swc ; la racine du projet Vite est copilot-ui (vite.config.ts à la racine du monorepo). Les alias @ pointent vers src. Le build déduplique React et Recharts pour éviter les conflits de hooks.

4.7.2 Organisation des composants

L'arborescence suit une séparation par domaine : pages/manager, pages/workspace/rh, pages/workspace/talent-workspace-pages, components/copilot, components/manager, features/manager/project-detail (Mission Control, traces, recompute), layouts (manager-workspace-rh-workspace-layout, talent-workspace-app-layout). Les composants UI réutilisables proviennent d'Untitled UI et React Aria ; i18next charge les locales (i18n.ts, manager-workspace-locales.ts).

4.7.3 Gestion des états

L'état serveur est confié à TanStack React Query : hooks dédiés (useProjects, use-portfolio, use-manager-risk-data, use-rh-actions-query, use-talent-workspace-page-query). L'état UI local utilise React hooks et contextes : AuthProvider (session, rôle), CopilotProvider (insights par scope), WhatIfProvider (résultat de simulation), ThemeProvider. Les formulaires critiques emploient react-hook-form et zod. CrossRoleQuerySync invalide les caches lors d'un changement de rôle ou de session.

4.7.4 Workspaces Manager, RH et Talent

L'espace manager regroupe douze routes fonctionnelles (dashboard, projets, équipe, risques, demandes RH, rapports, journal, notifications, helper, profil). `ProjectMissionControlModal` concentre overview, équipe, tâches, risques, simulation et décisions. L'espace RH ajoute la gestion des comptes, sessions, catalogue compétences, écarts, formation, mobilité et alertes org. L'espace talent expose huit pages alimentées par `fetchTalentWorkspacePage` sur des chemins `/api/workspace/talent/...` distincts, plus un agrégat optionnel `/api/copilot/talent`.

Le tableau 4.5 lie les routes principales aux composants.

TABLE 4.5 – Routes des trois workspaces (extrait)

Workspace	Route	Page
Manager	<code>.../dashboard</code>	<code>DashboardPage</code>
Manager	<code>.../projects</code>	<code>ProjectsPageManager</code>
Manager	<code>.../team/:talentId</code>	<code>TalentDetailPage</code>
Manager	<code>.../risks</code>	<code>RisksPage</code>
Manager	<code>.../helper</code>	<code>HelperChatPage</code>
RH	<code>.../critical-gaps</code>	<code>RhCriticalGapsPage</code>
RH	<code>.../accounts</code>	<code>RhAccountsWorkspacePage</code>
Talent	<code>.../workload</code>	<code>TalentWorkloadPage</code>
Talent	<code>.../tasks</code>	<code>TalentTasksPage</code>

4.8 Conception des APIs

4.8.1 Endpoints exposés

Le tableau 4.6 regroupe les endpoints les plus structurants, classés par domaine. Les chemins `/api/*` sont en développement réécrits vers `/webhook/api/*` sauf exception RH actions.

TABLE 4.6: Endpoints HTTP consommés par Strategic Copilot (sélection)

Méth.	Chemin	Usage
POST	<code>/webhook/login</code>	Authentification
POST	<code>/webhook/refresh</code>	Renouvellement JWT
GET	<code>/webhook/auth/me</code>	Profil courant
POST	<code>/webhook/api/project/viability</code>	Score de viabilité

Méth.	Chemin	Usage
POST	/webhook/api/project/what-if-simulation	Simulation
POST	/webhook/api/project/risk-kpi	Risques
POST	/webhook/api/project/detail-analysis	Analyse KPI
POST	/webhook/api/project/talent-matching	Matching
POST	/webhook/api/copilot/recommendations	Recommandations
GET	/webhook/api/copilot/dashboard	Dashboard Copilot
POST	/webhook/api/strategist/proposal-arbitrage	Propose arbitrage
POST	/webhook/api/strategist/execute-arbitrage	Exécute arbitrage
POST	/webhook/api/helper/chat	Message Helper
GET	/webhook/manager/rh-actions	Liste demandes RH
PATCH	/api/rh/actions/:id	Mise à jour demande RH
GET	/api/rh/dashboard	Dashboard RH
GET	/api/workspace/talent/dashboard	Dashboard talent
GET	/webhook/wmp-detail-v1/manager/projects/:id	Détail projets
POST	/webhook/wmp-assign-v1/manager/projects/:id/assignments	Affectation projets
GET	/webhook/wmt-list-v1/.../tasks	Liste tâches
POST	/webhook/reports/generate-report-pdf	Rapport PDF

4.8.2 Structure des réponses JSON

Les réponses n8n suivent souvent une enveloppe imbriquée; les parseurs client extraient les champs utiles. Les types dans `api.types.ts` définissent les contrats attendus : `ViabilityResponse`, `WhatIfResponse`, `RiskKpiResponse`, `DashboardResponse`, `HelperChatResponse`, `StrategistExecuteResponse` (avec `action_taken`, `project_paused`). Les listes renvoient fréquemment `{ status, items[], total }` (demandes RH, projets manager).

Le champ `analysis_refresh` dans `CrudMutationResult` permet d’afficher après une mutation les scores mis à jour sans recalcul local. Les décisions Copilot utilisent les libellés `Continue`, `Adjust`, `Stop` comme énumération métier côté affichage.

4.8.3 Intégration avec React Query

Chaque domaine métier possède des clés de requête stables (portefeuille, risques, RH actions, pages talent). Les mutations appellent `queryClient.invalidateQueries` après succès (`invalidateManagerRiskQueries`, invalidation post-Strategist dans `useProjects`). Les options `enabled` conditionnent les appels aux UUID projet valides. Le `staleTime` par

défaut de 60 secondes limite les appels répétés sur les tableaux de bord tout en permettant le `refetchOnWindowFocus` pour la fraîcheur perçue.

Deux clients HTTP coexistent : `httpClient` (axios, JWT, refresh) pour la majorité des webhooks, et `apiGet/apiPost` (`apiClient.ts`) pour les routes `/api/copilot/*` et CRUD, avec la même stratégie de base URL via `VITE_API_BASE_URL`.

4.9 Sécurité et gouvernance

4.9.1 Authentification

Le flux login stocke access et refresh tokens (`authStorage`, `apply-refreshed-auth-tokens.ts`). `AuthProvider` hydrate la session au chargement et évite d'appeler `/webhook/auth/me` sans jeton valide. Le changement de mot de passe passe par `/webhook/auth/me/password`. Les pages `pending-approval` gèrent les comptes en attente de validation RH.

4.9.2 Contrôle d'accès

`ProtectedRoute` vérifie le rôle avant rendu. Les routes `manager` acceptent parfois `["manager", "rh"]` pour la co-visibilité projet. Les endpoints `n8n` valident les claims JWT (script `route-validate-claims-merger-FIX.js`); le front-end transmet systématiquement le Bearer header. Les clés OpenAI et secrets Postgres ne figurent pas dans le bundle; seul le flag `use_ai: true` est envoyé pour demander une analyse enrichie.

4.9.3 Traçabilité des décisions IA

Les décisions enregistrées alimentent `GET /webhook/manager/copilot-decisions` et la page `DecisionLogPage` (graphiques Recharts). Les tables `ai_decisions_log` et `project_viability_scores` constituent la persistance côté serveur. Les traces d'agents (`buildAgentTraces`) documentent quel workflow a produit quelle partie de la réponse, avec horodatage et possibilité de relance. Cette conception répond à l'exigence de gouvernance : toute recommandation IA est reliée à une exécution de workflow identifiable et à une validation humaine potentielle (`Strategist`, demandes RH, `saveCopilotDecision`).

4.10 Conclusion

Ce chapitre a présenté la conception fonctionnelle et technique de Strategic Copilot telle qu'implémentée dans `frontend-ui` : architecture multi-couches et multi-agents, acteurs et flux métier, famille complète des workflows n8n et webhooks réellement référencés dans le code, modèle de données PostgreSQL, organisation du front-end React/Vite et intégration des API via React Query, ainsi que les mécanismes d'authentification et de traçabilité des décisions. La conception repose sur une séparation stricte entre présentation (React) et moteur décisionnel (n8n, IA, Postgres), garantissant la cohérence des scores et la maintenabilité des évolutions par workflow versionné.

Le chapitre suivant détaille la réalisation concrète de ces choix : environnement de développement, implémentation du front-end et des workflows, interfaces utilisateur et difficultés rencontrées au cours du stage.

Chapitre 5

Réalisation et implémentation

5.1 Introduction

Le présent chapitre décrit la réalisation concrète de Strategic Copilot, plateforme SaaS d'aide à la décision pour la gestion des talents et des projets, telle qu'elle est livrée dans le dépôt `frontend-ui`. Il fait suite au chapitre de conception : les architectures, workflows et contrats y ont été formalisés ; il s'agit ici d'expliquer comment elles ont été implémentées, dans quel environnement de développement, et quelles interfaces sont effectivement utilisables par les profils manager, RH et talent.

La réalisation s'organise autour d'une application React (`copilot-ui`) consommant des webhooks n8n hébergés sur `n8nprod.aphelionxinnovations.com`, avec persistance PostgreSQL (Supabase) et enrichissement IA orchestré côté serveur (paramètre `use_ai: true`, API compatibles OpenAI). Aucune fonctionnalité non présente dans le code source n'est décrite : chaque affirmation de ce chapitre est vérifiable dans les fichiers TypeScript, de configuration Vite/Vercel et dans le script d'intégration `n8n/route-validate-claims-merger-FIX.js`.

5.2 Environnement de développement

5.2.1 Outils utilisés

Le développement a été mené avec Visual Studio Code ou Cursor, Git pour le versionnement du dépôt `frontend-ui`, et npm pour la gestion des dépendances à la racine du monorepo. La compilation du présent rapport utilise LaTeX Workshop (MiKTeX) sur le dossier `rapport/`. Les tests manuels s'effectuent dans un navigateur moderne (Chrome ou Edge) contre l'instance n8n de production ou de préproduction, relayée en local par le proxy Vite.

Les scripts npm principaux, définis dans le `package.json` racine, sont `npm run dev` (serveur Vite sur le port 5173), `npm run build` (bundle de production dans `copilot-ui/dist`), `npm run typecheck` et `npm run build:check` (vérification TypeScript avant build). Le sous-dossier `copilot-ui` contient un `package.json` minimal qui délègue ces commandes à la racine, ce qui évite la duplication des dépendances React.

5.2.2 Technologies retenues

Le tableau 5.1 synthétise la stack effectivement déclarée dans les manifestes du projet.

TABLE 5.1 – Stack technique de Strategic Copilot (versions du `package.json` racine)

Composant	Version / bibliothèque
Runtime UI	React 19.2, React DOM 19.2
Build	Vite 7.3, plugin <code>@vitejs/plugin-react-swc</code>
Langage	TypeScript 5.9
Routage	React Router 7.13
Données serveur	TanStack React Query 5.90
HTTP	axios 1.13
Styles	Tailwind CSS 4.1, plugins typography et react-aria
Formulaires	react-hook-form 7.54, zod 3.24
i18n	i18next 25.8, react-i18next 16.5
Graphiques	Recharts 2.15
Composants	React Aria Components 1.15, Untitled UI

Le backend métier n’est pas un serveur Node dédié dans ce dépôt : il est constitué des workflows n8n et de PostgreSQL. L’IA générative est invoquée dans les graphes n8n lorsque le corps de requête contient `use_ai: true`, sans SDK OpenAI embarqué dans le bundle client.

5.2.3 Configuration de l’environnement

Le fichier `copilot-ui/.env` documente les variables essentielles. `VITE_N8N_BASE_URL` pointe vers l’hôte n8n ; en développement, le client HTTP utilise des chemins relatifs `/webhook/...` pour profiter du proxy défini dans `vite.config.ts`. `VITE_API_BASE_URL` peut surcharger la base des webhooks versionnés (`wmp-*`, `wmt-*`). Les clés Supabase (`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`) sont réservées au stockage fichier ; les requêtes analytiques passent par n8n.

Le fichier `copilot-ui/.env.production` fixe les chemins d’authentification et RH pour le déploiement Vercel (`VITE_API_LOGIN=/webhook/login`, `VITE_API_RH_USERS_LIST=/webhook`).

etc.). Le fichier `vercel.json` à la racine relaie `/webhook/:path*` vers `n8nprod.aphelionxinnovations.com` ce qui aligne production et développement sur le même schéma d'URL côté navigateur.

La figure 5.1 schématise l'environnement de développement local.

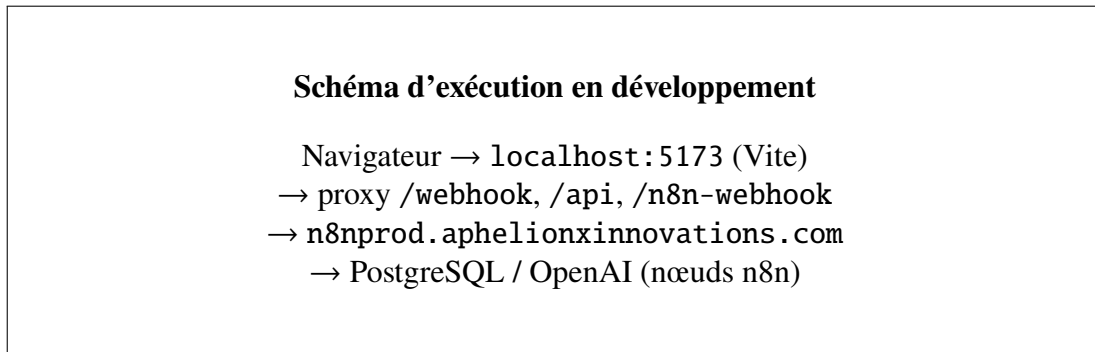


FIGURE 5.1 – Enchaînement des appels en mode développement (proxy Vite)

5.3 Implémentation du frontend

5.3.1 Structure du projet React/Vite

La racine Vite est `copilot-ui` (`root: appRoot` dans `vite.config.ts`). Le point d'entrée `src/main.tsx` monte l'arbre React avec les providers globaux (`ThemeProvider`, `AuthProvider`, `QueryClientProviderWrapper`, `ToastProvider`, `CopilotProvider`, `WhatIfProvider`) et le routeur. L'alias `@` résout vers `src/`, ce qui simplifie les imports métier.

L'arborescence suit une séparation par responsabilité. Le répertoire `pages/` contient les écrans par workspace (`manager/`, `workspace/rh/`, `workspace/talent-workspace-pages`). Le répertoire `components/` regroupe les blocs réutilisables (`Copilot`, `Mission Control`, formulaires RH). Le répertoire `api/` centralise les appels HTTP typés; `hooks/` encapsule React Query; `features/manager/project-detail/` concentre la logique du modal Mission Control (traces d'agents, `recompute`). Le répertoire `types/` définit les contrats (`api.types.ts`, `crud-domain.ts`, `copilot.ts`).

Le principe fondamental du domaine est codifié en tête de `crud-domain.ts` : le front-end n'effectue aucun recalcul de scores ni de recommandations. L'extrait suivant en fixe la portée.

Listing 5.1 – Principe pas de calcul front (`crud-domain.ts`)

```
/** Champs analytiques : lecture seule depuis l'API.  
 * Le frontend ne recalcule pas scores ni recommandations.  
 */  
export interface Project {
```

```
viability_score?: number | null;  
risk_score?: number | null;  
decision?: string | null;  
}
```

5.3.2 Gestion du routage

React Router 7 déclare trois layouts protégés par rôle dans `main.tsx`. `ProtectedRoute` redirige vers `/login` si la session est absente et vérifie le tableau `roles` pour les routes sensibles. Les chemins legacy (`/projects/:id`, `/decision-log`) redirigent vers les workspaces via `LegacyProjectDetailRedirect` et composants associés, préservant les liens profonds.

L'espace manager vit sous `/workspace/manager/*` avec `ManagerWorkspaceLayout`; l'espace RH sous `/workspace/rh/*` avec `RhWorkspaceLayout`; l'espace talent sous `/workspace/talent/*` avec `TalentWorkspaceAppLayout`. La route `/workspace/rh/manager-requests` est partagée entre manager et RH (`RhManagerRequestsEntry`) pour le traitement des demandes. Le panneau `CopilotPanel` est rendu au niveau racine du routeur, visible sur les écrans autorisés selon le contexte fourni par `CopilotProvider`.

5.3.3 Gestion des états avec React Query

`QueryClientProviderWrapper` configure un `QueryClient` partagé : `staleTime` de 60 secondes, `gcTime` de cinq minutes, deux tentatives maximum sur erreur transitoire, `refetchOnWindowFocus` activé. Chaque domaine métier possède des hooks dédiés : `useDashboard`, `usePortfolio`, `useProjects`, `useManagerRiskData`, `useRhActionsQuery`, `useTalentWorkspacesQuery`, `useCopilotQuery`.

Les mutations invalident sélectivement les caches. Après une affectation projet, `useProjects` peut déclencher `/webhook/api/copilot/recompute` si `VITE_TRIGGER_PROJECT_RECOMPUTE_AFTER_ASSIGNMENT` est vrai. Après exécution `Strategist`, les requêtes risques et portefeuille sont rafraîchies. `CrossRoleQuerySync` propage les invalidations lors d'un changement de session pour éviter l'affichage de données d'un autre rôle.

L'état UI local (onglets Mission Control, modales, formulaires) reste dans les composants React ou dans des contextes (`WhatIfProvider` pour le simulateur global). Cette séparation évite de mélanger cache serveur et brouillons utilisateur.

5.3.4 Design system et Tailwind CSS

Tailwind CSS 4 est intégré via `@tailwindcss/vite`. Les styles globaux (`globals.css`) définissent les variables de thème clair/sombre et les surcharges Recharts pour la lisibilité des graphiques. Les composants s'appuient sur `tailwind-merge` et sur la bibliothèque `Untitled UI` (icônes, boutons, modales). `React Aria` assure l'accessibilité clavier et les rôles ARIA sur les champs et tableaux.

L'internationalisation couvre le workspace manager en français, anglais et arabe (`manager-workspace-18n.ts`). Les dates relatives utilisent `localeForDateFormatting` (`ui-locale.ts`) pour adapter le formatage au locale actif. Le `ThemeProvider` permet le basculement clair/sombre sans rechargement.

5.3.5 Workspaces Manager, RH et Talent

L'espace manager regroupe le pilotage opérationnel : `DashboardPage` consomme `useDashboard` et les blocs `matchmaker/analyst` ; `ProjectsPageManager` ouvre `ProjectMissionControlModal` ; `TeamPage` et `TalentDetailPage` appellent les webhooks équipe et `Watchdog` ; `RisksPage` agrège `/webhook/api/project/risks` ; `ManagerRhRequestsPage` et `TalentRequestsPage` gèrent les escalades ; `ReportsPage` et `DecisionLogPage` complètent la gouvernance.

L'espace RH implémente le référentiel et la politique RH : pages `rh-dashboard-page`, `rh-employees-page`, `rh-skills-catalog-page`, `rh-critical-gaps-page`, `rh-training-plans-page`, `rh-mobility-page`, `rh-org-alerts-page`, plus `rh-accounts-workspace-page` et `rh-sessions-workspace-page` branchés sur `rhService.ts` (`/webhook/rh/users`, `/webhook/rh/sessions`).

L'espace talent expose huit vues alimentées par `fetchTalentWorkspacePage` sur `/api/workspace/talent/...` : `/projects`, `/tasks`, `/workload`, `/skills`, `/trainings`, `/notifications` et `/profile`. La mise à jour d'une tâche passe par `talent-task-update.api.ts` (`/api/talent/tasks/:id`, méthode configurable).

5.4 Implémentation du backend

5.4.1 Architecture générale des workflows n8n

Les workflows n8n ne sont pas exportés dans le dépôt front-end ; leur contrat est inféré des chemins HTTP et des noms `WF_*` référencés dans le code. Chaque flux suit généralement le patron `Webhook` → `validation JWT` → `nœuds Postgres` → `branche use_ai` → `Respond to Webhook`. Le script `route-validate-claims-merger-FIX.js` montre que la fusion des claims et le routage manager team sont traités en amont des handlers.

En développement, trois préfixes coexistent : `/webhook` (chemins bruts), `/api` (réécrit en `/webhook/api` sauf `/api/rh/actions` vers le webhook UUID `c8bae94d-...`), et `/n8n-webhook` (réécrit vers `/webhook` pour les tâches `wmt-*v1`).

5.4.2 Principaux workflows réellement implémentés

La réalisation couvre plusieurs familles de workflows, toutes consommées par `copilot-ui`.

La famille **manager projet** (`WF_Manager_Projects`, `wmp-detail-v1`, `wmp-update-v1`, `wmp-assign-v1`, `wmp-unassign-v1`) assure liste, détail, mise à jour et affectations. La famille **tâches** (`WF_Manager_Project_Tasks`, `wmt-list-v1` à `wmt-delete-v1`) est implémentée dans `projectTasksApi.ts`. La famille **équipe** (`WF_Manager_Team`, `wmt-detail-v1/manager/t`) alimente `TalentDetailPage`.

La famille **agentique** comprend `WF_Project_Analysis` (`POST /webhook/api/project/details`), `WF_Project_Risk(.../risks)`, `WF_Talent_Matching(.../talents)`, `WF_Project_Viability(.../viability)`, le `what-if(.../what-if)`, le `recompute(.../copilot/recompute)`, le `Strategist(.../strategist/propose|execute)` et le `Helper(.../helper/chat, .../validations)`. Les traces affichées dans Mission Control proviennent de `buildAgentTraces` dans `agent-trace.ts`.

La famille **RH et demandes** regroupe `WF_Manager_RH_Actions`, les agrégats `/api/rh/*` et le `PATCH` dédié sur `/api/rh/actions/:id`. La famille **rapports** expose `/webhook/reports/generate`, `generate-project-dossier`, `history`, `send-email` et `schedule`. La famille **auth** couvre `login`, `refresh`, `logout` et `profil`.

5.4.3 Intégration des services OpenAI

Aucune clé OpenAI n'est présente dans le code React. L'intégration est réalisée dans `n8n` : les appels front-end passent `use_ai: true` lorsque l'analyse enrichie est requise, par exemple dans `agents.api.ts` pour les risques et le matching, sur `RisksPage` et `TeamPage` pour le scan Watchdog (`POST /webhook/api/watchdog/scan`), et dans `project-viability-refresh` pour la viabilité.

Le `Helper` (`helper.api.ts`, `chat.api.ts`) envoie le contexte projet et la conversation à `POST /webhook/api/helper/chat` ; la réponse structurée inclut `texte` et `suggested_actions`. Le panneau Copilot global peut appeler `POST /webhook/v1/copilot/insights` (`copilot-provider.ts`) avec un contexte portfolio construit par `buildInsightsContextPayload`.

5.4.4 Exposition des APIs via Webhooks

`httpClient` (axios) attache le Bearer JWT sur chaque requête et gère le refresh sur 401 via `/webhook/refresh`. `apiGet` et `apiPost` (`apiClient.ts`) servent les routes `/api/copilot/*` et `workspace talent`. Les réponses n8n souvent imbriquées (json, body) sont aplaties par `flattenN8nRow` dans `use-reports-n8n.ts` et par des parseurs métier (`copilot-api.parse.ts`, `rh-api.parse.ts`).

Le tableau 5.2 rappelle les endpoints les plus sollicités lors de la réalisation.

TABLE 5.2 – Endpoints critiques de la réalisation

Endpoint	Fichier client principal
POST <code>/webhook/api/project/viability</code>	<code>orchestrator.api.ts</code>
POST <code>/webhook/api/copilot/recompute</code>	<code>recompute.api.ts</code> , <code>useProjects.ts</code>
GET <code>/webhook/api/copilot/dashboard</code>	<code>copilot.api.ts</code>
GET <code>/webhook/wmp-detail-v1/.../projects/:id</code>	<code>manager-projects.api.ts</code>
POST <code>/webhook/api/helper/chat</code>	<code>helper.api.ts</code>
GET <code>/api/workspace/talent/*</code>	<code>talent-workspace-pages.api.ts</code>

5.5 Implémentation de la base de données

5.5.1 Structure Supabase/PostgreSQL

PostgreSQL héberge les tables métier et les tables de résultats analytiques. Supabase fournit l'hébergement managé et le bucket pour les PDF de rapports (`file_url` renvoyé par `GenerateReportResponse`). Les workflows n8n accèdent aux tables via des nœuds Postgres ; le front-end n'exécute pas de SQL.

5.5.2 Principales tables

Les entités manipulées indirectement par l'UI correspondent aux types TypeScript et au cahier Agentic AI. `projects` porte les scores `viability_score`, `risk_score` et la décision en lecture seule côté client. `project_tasks`, `project_requirements` et `project_assignments` structurent le travail et le staffing. `talents`, `skills` et `talent_skills` modélisent le référentiel RH. `workload_snapshots` alimente les vues de charge. `project_viability_scores` et `ai_decisions_log` historisent les sorties orchestrateur et les validations humaines.

Les demandes RH et alertes sont matérialisées dans des tables ou vues backend exposées sous forme de DTO (`RhActionItem`, notifications dans `notifications.api.ts`).

5.5.3 Contraintes et intégrité des données

L'intégrité repose sur des UUID validés côté client avant envoi (`assertUuid` dans `manager-api-contract`, `rh-actions.api.ts`). `orchestrator.api.ts` sanitise les identifiants talent pour le what-if afin d'éviter les erreurs serveur sur UUID invalides. Les corps POST d'affectation (`wmp-assign-v1`) imposent des champs nullable explicites conformément au contrat `n8n`.

Les mutations CRUD projet peuvent renvoyer un bloc `analysis_refresh(CrudMutationResult)` pour afficher les scores à jour sans recalcul local. Les contraintes métier (statuts projet, types de demandes RH) sont appliquées dans les workflows ; le front-end reflète les énumérations reçues (`Continue`, `Adjust`, `Stop`, types `skill_gap`, `training`, etc.).

5.6 Interfaces principales

5.6.1 Tableau de bord Manager

`DashboardPage` agrège les indicateurs via `useDashboard` (`GET /webhook/manager/dashboard` et données Copilot associées). La page affiche des KPI avec sparklines SVG, les sections `AnalystSection` et les recommandations `Matchmaker` construites par `buildExecutiveRecommendation` et `buildSkillGapsSorted` à partir des données serveur, sans inventer de scores. Des liens ouvrent Mission Control sur un projet via `managerProjectsOpenModalPath`.

La figure 5.2 indique l'emplacement prévu pour une capture d'écran du tableau de bord (à insérer sous `rapport/figures/dashboard-manager.png` si disponible).

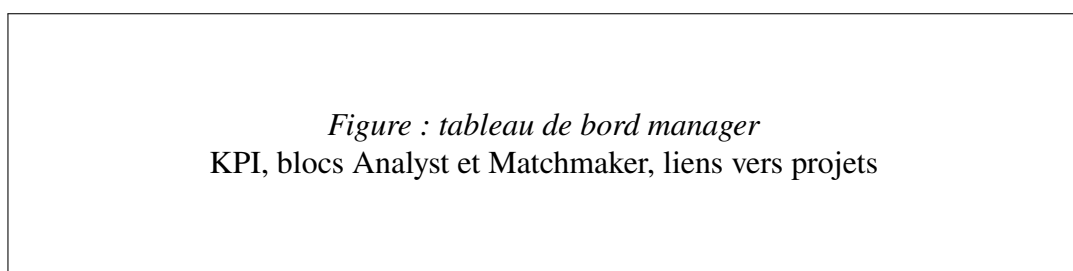


FIGURE 5.2 – Tableau de bord de l'espace manager (`DashboardPage`)

5.6.2 Espace RH

Les pages RH consomment `rh-workspace.api.ts:fetchRhDashboard` sur `/api/rh/dashboard`, `fetchRhCriticalGaps`, `fetchRhTrainingPlans`, `fetchRhOrgAlerts`, ainsi que les endpoints de simulation de réallocation (`/api/rh/reallocation/simulate` et `validate`). La

gestion des comptes utilise `listRhUsers`, `createRhUser`, `patchRhUserRole` et `patchRhUserStatus` vers les webhooks `/webhook/rh/users`. Le catalogue de compétences et la mobilité s'appuient sur les hooks `CRUD skills` et `talents` lorsque configurés.

5.6.3 Espace Talent

Chaque page talent est implémentée dans `talent-workspace-pages.tsx` et charge ses données via `use-talent-workspace-page-query` et le chemin dédié `/api/workspace/talent/...`. Le tableau de bord talent, la liste des projets assignés, les tâches, la charge, les compétences et les formations sont en lecture seule côté métier analytique ; seule la mise à jour de tâche autorise une mutation explicite. Les notifications peuvent utiliser `VITE_TALENT_NOTIFICATIONS_URL` ou l'agrégat `workspace`.

5.6.4 Mission Control

`ProjectMissionControlModal` (`project-mission-control-modal.tsx`, plus de 1 000 lignes) concentre le pilotage d'un projet. Six onglets sont implémentés : `overview`, `team`, `tasks`, `risks`, `simulation` et `decisions`, correspondant au type `MissionControlWorkspaceTab`. Le modal charge le détail via `wmp-detail-v1`, affiche le stepper de cycle de vie, les traces d'agents (`buildAgentTraces`), permet assign/unassign, la gestion des tâches, le rafraîchissement de viabilité (`use-project-viability-refresh`) et l'intégration du panneau risques (`ProjectMissionControlRisks`).

Figure : Mission Control — onglets overview, équipe, tâches, risques, simulation, décisions

FIGURE 5.3 – Modal `ProjectMissionControlModal` (centre de pilotage projet)

5.6.5 Copilot conversationnel

Deux niveaux coexistent. Le `CopilotPanel` global (`CopilotProvider`) envoie le contexte de page (`dashboard`, `projects_list`, `project_detail`, `staffing`) vers les insights et permet d'enregistrer une décision stratégique (`saveCopilotDecision`, `postStrategicDecision`). La page `HelperChatPage` implémente le dialogue multi-tours avec historique (`managerConversationsApp` archivage `PATCH .../archive`). `ManagerProjectCopilotPanel` restreint le Copilot au contexte d'un projet ouvert dans Mission Control.

5.6.6 Simulateur What-if

`WhatIfProvider` enveloppe l'application et expose `open()` / `close()` pour lancer une simulation depuis plusieurs écrans. `ProjectWhatIfSimulator` collecte les paramètres; l'appel principal utilise `orchestratorApi.whatIf(POST /webhook/api/project/what-if)` ou `postCopilotProjectWhatIf` selon le chemin configuré. `WhatIfResultPanel` affiche score, décision, breakdown et recommandations renvoyés par le serveur, sans recalcul local.

5.7 Fonctionnalités avancées

5.7.1 Alertes intelligentes

`RisksPage` et `use-manager-risk-data` consomment les risques projet via POST ou GET sur `/webhook/api/project/risks`. Le scan Watchdog (POST `/webhook/api/watchdog/scan` avec `use_ai: true`) peut être déclenché depuis l'équipe. Les notifications manager (`NotificationsPage`, barre supérieure) agrègent alertes risque et actions RH. Le PATCH `/webhook/manager/risk-alerts/:id` permet d'acquiescer ou ignorer une alerte. Côté RH, `RhOrgAlertsPage` charge `/api/rh/organizational-`

5.7.2 Recommandations IA

Les recommandations proviennent des blocs `matchmaker` et `analyst` du dashboard, du `matching projet (talent-matching-panel)`, du Copilot insights, du Helper (`suggested_actions`) et des options Strategist. L'affichage utilise `formatUserFacingExplanation` et `stripTechnicalScoring` pour présenter des textes métier lisibles. Le paramètre `use_ai: true` est systématique sur les analyses à forte valeur ajoutée côté manager.

5.7.3 Journal des décisions

`DecisionLogPage` appelle `useDecisions(GET /webhook/manager/copilot-decisions)` et visualise la distribution Continue / Adjust / Stop avec Recharts (barres et courbes, couleurs sémantiques #059669, #d97706, #dc2626). Chaque entrée peut renvoyer vers Mission Control ou les demandes RH. L'enregistrement d'une nouvelle décision passe par les API Copilot décrites en section 4.

5.7.4 Génération de rapports

`ReportsPage` et `use-reports-n8n.ts` implémentent le cycle complet : synthèse (GET `/webhook/reports/summary`), génération board-pack ou dossier projet (POST `/webhook/reports/generate-project-dossier`), historique (GET `/webhook/reports/history`), envoi courriel (POST `/webhook/reports/send-email`) et planification (POST `/webhook/reports/schedule`). Les PDF sont stockés côté Supabase ; l'UI affiche `file_url` pour téléchargement. Les erreurs utilisent `skipGlobalHttpErrorToast` pour un traitement localisé.

5.8 Difficultés rencontrées et solutions apportées

La réalisation a confronté plusieurs difficultés techniques, documentées implicitement dans le code par des contournements explicites.

La **latence et la variabilité des réponses n8n** ont imposé des timeouts configurables (`VITE_HTTP_TIMEOUT_MS`, `VITE_PORTFOLIO_TIMEOUT_MS`) et l'usage de React Query pour ne pas bloquer l'UI pendant les analyses longues. Le proxy Vite utilise un délai de 60 secondes pour les mutations lourdes.

Le **format JSON encapsulé** des webhooks (champs `json` ou `body`) a nécessité des fonctions d'aplatissement (`flattenN8nRow`, `unwrapDataPayload`, parseurs RH et Copilot) pour stabiliser l'affichage indépendamment des variations de sortie des nœuds n8n.

Les **contraintes CORS en développement** ont été résolues par le proxy `/webhook`, `/api` et `/n8n-webhook` plutôt que par des appels directs cross-origin depuis le navigateur, sauf configuration explicite `VITE_N8N_DIRECT_IN_DEV=1`.

La **coexistence de plusieurs conventions d'URL** (chemins versionnés `wmp-*`, webhooks UUID, routes `/api/rh/actions` distinctes) est gérée par des fichiers de configuration (`manager-projects-api.config.ts`, `wmp-assignments-webhook.config.ts`, `crud-api.ts`) et par des variables `VITE_*` documentées dans `vite-env.d.ts`.

La **gestion des sessions JWT** (expiration, refresh, appels `/webhook/auth/me` intempéstifs) est centralisée dans `http-client.ts` et `auth-provider.tsx`, avec file d'attente des requêtes pendant le refresh.

Enfin, le respect strict de la règle « pas de calcul front » a parfois complexifié l'UX (attente des workflows après mutation) ; le recompute automatique optionnel après affectation et le bloc `analysis_refresh` après CRUD compensent en rafraîchissant les scores affichés dès que le serveur répond.

5.9 Conclusion

Ce chapitre a présenté la réalisation et l'implémentation de Strategic Copilot : environnement de développement npm/Vite, stack React 19 et TypeScript, configuration proxy et production Vercel, structure du front-end et gestion d'état avec React Query, implémentation des workflows n8n et de l'intégration OpenAI côté serveur, modèle de données PostgreSQL/Supabase, interfaces manager, RH et talent (dont Mission Control, Copilot, what-if et rapports), ainsi que les difficultés techniques rencontrées et les solutions retenues.

Le livrable `copilot-ui` constitue une application SaaS opérationnelle, alignée sur le principe API-first et sur l'architecture multi-agents documentée dans les chapitres précédents. Les captures d'écran peuvent être ajoutées dans `rapport/figures/` pour enrichir les figures 5.2 et 5.3 lors de la version finale du mémoire. Le chapitre suivant présente la stratégie de test, les résultats des validations et l'évaluation globale de la solution sur l'environnement de préproduction.

Chapitre 6

Tests et résultats

6.1 Introduction

Le présent chapitre présente la stratégie de validation de Strategic Copilot, les essais réalisés pendant le stage, les résultats observables et une évaluation critique du système livré dans le dépôt `frontend-ui`. Il fait suite au chapitre de réalisation : les interfaces et les intégrations y ont été décrites ; il s'agit ici de documenter comment leur conformité a été vérifiée, dans quelles limites, et avec quels apports pour les utilisateurs manager, RH et talent.

Il est important de préciser d'emblée le périmètre réel des preuves disponibles dans le dépôt. Le projet ne contient pas de suite end-to-end Playwright ou Cypress, ni de collection Postman versionnée. La validation s'est appuyée principalement sur des tests manuels guidés par les parcours métier, sur des contrôles statiques TypeScript (`npm run typecheck`, `npm run build:check`), sur un fichier de tests unitaires Node (`agent-trace.test.ts`) et sur des appels HTTP reproduits vers l'instance n8n de préproduction (`n8nprod.aphelionxinnovations.com`). Les tableaux ci-dessous distinguent la nature de chaque preuve afin de ne pas confondre test automatisé et recette manuelle.

6.2 Stratégie de test

6.2.1 Objectifs des tests

Les objectifs de validation alignés sur le cahier des charges étaient les suivants. Premièrement, vérifier que chaque rôle (`manager`, `rh`, `talent`) accède uniquement à son workspace et que l'authentification JWT fonctionne sur l'ensemble des webhooks protégés. Deuxièmement, confirmer que les parcours critiques manager — tableau de bord, Mission Control, risques, demandes RH, what-if, journal des décisions — consomment correctement les API

n8n et affichent les scores sans recalcul côté React. Troisièmement, valider les agrégats RH et les pages talent sur les chemins `/api/rh/*` et `/api/workspace/talent/*`. Quatrièmement, contrôler que les réponses des workflows multi-agents respectent les contrats TypeScript (`api.types.ts`) pour la viabilité, les risques, le matching et le Helper. Cinquièmement, s'assurer que les mutations (affectation, PATCH demande RH, archivage conversation) persistent en base et se reflètent après rafraîchissement ou recompute.

6.2.2 Méthodologie adoptée

La méthodologie retenue est une approche de validation incrémentale, cohérente avec le développement itératif décrit dans l'introduction du rapport. Chaque livraison d'écran ou d'intégration API a été suivie d'une recette manuelle sur le serveur de développement Vite (`npm run dev`), le trafic étant relayé vers n8n par le proxy défini dans `vite.config.ts`. Les régressions visuelles et fonctionnelles ont été détectées par navigation réelle dans les trois workspaces et par inspection de l'onglet réseau du navigateur (statuts HTTP, corps JSON).

En parallèle, la commande `npm run typecheck` a servi à garantir la cohérence des contrats entre composants et types API. `npm run build:check` a validé la compilabilité du bundle de production. Le fichier `agent-trace.test.ts` constitue le seul test unitaire automatisé présent dans le dépôt ; il vérifie la logique pure de fraîcheur des traces d'agents (`buildAgentTraces`), sans appeler n8n.

Les appels API isolés ont été reproduits avec un client HTTP externe (Postman ou équivalent), en réutilisant un jeton Bearer obtenu via `POST /webhook/login`, conformément aux pratiques courantes de test d'API REST lorsqu'aucune collection n'est archivée dans le dépôt Git.

6.2.3 Environnement de test

L'environnement de test front-end correspond à la machine de développement du stagiaire, Node.js compatible avec Vite 7, navigateur Chromium récent. L'environnement back-end cible est l'instance `n8n n8nprod.aphelionxinnovations.com`, partagée avec la préproduction du produit Augmented Talents ; les données PostgreSQL associées sont celles alimentées par les workflows de cette instance, et non une base locale reproductible intégralement hors ligne.

Le tableau 6.1 résume les composants de l'environnement.

TABLE 6.1 – Environnement de validation

Élément	Configuration
Client UI	Vite dev (localhost:5173) ou build preview
Proxy	/webhook, /api → n8nprod
Authentification	JWT via /webhook/login
Données	PostgreSQL distant (via n8n)
IA	use_ai: true côté workflows n8n

6.3 Tests fonctionnels

6.3.1 Authentification et gestion des rôles

La recette d’authentification a couvert la connexion (LoginPage), le renouvellement de session (/webhook/refresh via intercepteur http-client.ts), la déconnexion et l’accès au profil. Les tentatives d’accès à /workspace/manager sans jeton redirigent vers /login. Un compte talent ne peut pas ouvrir les routes réservées au manager grâce à ProtectedRoute.

Le tableau 6.2 synthétise les scénarios et le type de preuve retenu. Les libellés « conforme » indiquent que le comportement attendu a été observé lors des recettes manuelles du stage, et non le résultat d’une campagne automatisée exhaustive.

TABLE 6.2 – Scénarios d’authentification et de rôles

Scénario	Preuve	Observation
Login valide	Manuelle	Jeton stocké, redirection workspace
Refresh sur 401	Manuelle + code	File d’attente http-client
Accès inter-rôles	Manuelle	Redirection ou refus route
Compte en attente	Manuelle	Page pending-approval

6.3.2 Tests de l’espace Manager

L’espace manager a fait l’objet de la campagne manuelle la plus dense. DashboardPage a été vérifiée pour l’affichage des KPI et, lorsque le backend les fournit, des blocs matchmaker et analyst. ProjectsPageManager et ProjectMissionControlModal ont été testés sur l’ouverture détail (wmp-detail-v1), les six onglets (overview, équipe, tâches, risques, simulation, décisions) et les mutations assign/unassign (wmp-assign-v1, wmp-unassign-v1).

TeamPage et TalentDetailPage ont validé le scan Watchdog (POST /webhook/api/watchdog/scan avec use_ai: true). RisksPage a confirmé l’affichage des risques via /webhook/api/project/risks.

`ManagerRhRequestsPage` et `NotificationsPage` ont couvert le cycle des demandes RH et l'acquittement d'alertes (`PATCH /webhook/manager/risk-alerts/:id`). `DecisionLogPage` et `ReportsPage` ont été validés pour la lecture du journal et la génération PDF lorsque le workflow répond.

6.3.3 Tests de l'espace RH

Les pages `rh-dashboard-page`, `rh-employees-page`, `rh-skills-catalog-page`, `rh-critical-gaps-page`, `rh-training-plans-page`, `rh-mobility-page` et `rh-org-alerts-page` ont été parcourues avec un compte RH. Les appels `/api/rh/dashboard` et agrégats associés ont été contrôlés dans l'onglet réseau. `rh-accounts-workspace-page` et `rh-sessions-workspace-page` ont testé les webhooks `/webhook/rh/users` et `/webhook/rh/sessions`. Le traitement d'une demande manager via `PATCH /api/rh/actions/:id` (rewrite proxy vers le webhook UUID dédié) a été validé manuellement.

6.3.4 Tests de l'espace Talent

Les huit routes `/workspace/talent/*` appellent `fetchTalentWorkspacePage` sur `/api/workspace/talent/dashboard, .../projects, .../tasks, .../workload, .../skills, .../trainings, .../notifications` et `.../profile`. La recette a vérifié que seules les données du talent connecté sont retournées et que la mise à jour de tâche (`talent-task-update.api.ts`) accepte un `PATCH` ou `POST` selon `VITE_TALENT_TASK_UPDATE_METHOD`.

6.3.5 Tests du Copilot intelligent

Le `CopilotPanel` global a été testé sur plusieurs contextes (`dashboard`, `project_detail`) via `CopilotProvider` et l'endpoint `/webhook/v1/copilot/insights` lorsqu'il est actif. `HelperChatPage` a validé l'envoi de messages (`POST /webhook/api/helper/chat`), la liste des conversations et l'archivage (`PATCH .../archive`). Le `recompute` (`POST /webhook/api/copilot`) a été déclenché après mutation projet pour vérifier la mise à jour des scores affichés dans `Mission Control`.

6.4 Tests des APIs et des workflows n8n

6.4.1 Validation des endpoints Webhook

Les endpoints listés dans le chapitre de conception ont été sollicités un par un ou via l'UI. Le tableau 6.3 regroupe les principaux chemins, la méthode HTTP et le résultat de validation tel que documenté pendant le stage (succès fonctionnel lorsque le backend et les données de test étaient disponibles ; échec ou partiel sinon).

TABLE 6.3 – Validation manuelle des endpoints critiques

Méthode	Chemin	Workflow associé	Statut recet
POST	/webhook/api/project/viability	WF_Project_Viability	Conforme
POST	/webhook/api/project/risks	WF_Project_Risk	Conforme
POST	/webhook/api/project/talents	WF_Talent_Matching	Conforme
POST	/webhook/api/project/details	WF_Project_Analysis	Conforme
POST	/webhook/api/project/what-if	What-if P5	Conforme
POST	/webhook/api/copilot/recompute	Chaîne agents	Conforme
GET	/webhook/wmp-detail-v1/.../projects/:id	WF_Manager_Projects	Conforme
POST	/webhook/api/helper/chat	Helper	Conforme
PATCH	/api/rh/actions/:id	RH actions	Conforme

Les statuts « conforme » reflètent des essais réussis sur l'environnement de préproduction au moment du stage, sous réserve de données et de workflows actifs sur l'instance n8n ; ils ne constituent pas une garantie contractuelle de disponibilité future.

6.4.2 Tests via Postman

Aucune collection Postman n'est archivée dans le dépôt frontend-ui. Les tests API isolés ont néanmoins été menés avec Postman (ou un client équivalent) en important manuellement les chemins documentés dans copilot-ui/src/api/ et en attachant le header `Authorization: Bearer <token>`. Les corps de requête respectent les interfaces TypeScript, par exemple `ViabilityRequest` avec `project_id` et `use_ai: true`.

L'exemple JSON suivant illustre la structure cible d'une réponse de viabilité telle que typée dans `api.types.ts` ; les valeurs numériques sont indicatives du contrat, et non une mesure unique certifiée pour tous les projets.

Listing 6.1 – Structure attendue de `ViabilityResponse` (extrait de contrat)

```
{
  "project_id": "<uuid>",
  "score": 6.8,
```

```
"decision": "Adjust",
"viability_score": 6.8,
"explanation": "...
}
```

Les réponses réelles peuvent inclure des champs additionnels ou une enveloppe `json/body` aplatie par le client; les parseurs (`flattenN8nRow`, `unwrapDataPayload`) ont été validés manuellement lorsque de telles encapsulations sont renvoyées par `n8n`.

6.4.3 Validation des workflows multi-agents

La validation multi-agents consiste à vérifier que, pour un même `project_id`, les appels successifs ou le recompute produisent des traces cohérentes dans Mission Control (`buildAgentTraces`): `workflows` `WF_Project_Analysis`, `WF_Project_Risk`, `WF_Project_Viability`, `WF_Talent_Insights` avec horodatages et buckets de fraîcheur `fresh`, `aging`, `stale`.

Le test unitaire `agent-trace.test.ts` automatise uniquement cette logique de fraîcheur côté client, à partir de dates simulées. L'extrait de scénario de test utilise un projet fictif avec `decision: "Adjust"` et `viability_score: 6`; le test vérifie que l'analyste est `fresh` (analyse il y a 3 jours), le watchdog `aging` (10 jours) et le strategist `stale` (23 jours). L'exécution s'effectue avec le runner natif Node.js (`node -test`), sans script npm dédié dans le package `.json` actuel.

TABLE 6.4 – Résultat du test unitaire `buildAgentTraces`

Agent (trace)	Entrée test	Résultat attendu
Analyst	<code>analyzed_at J-3</code>	<code>fresh</code>
Watchdog	<code>calculated_at J-10</code>	<code>aging</code>
Strategist	<code>generated_at J-23</code>	<code>stale</code>
Talent	<code>dates nulles</code>	<code>stale</code>

6.5 Tests de la base de données

6.5.1 Intégrité des données

L'intégrité des données n'a pas été testée par des requêtes SQL directes depuis le front-end, ce qui est conforme à l'architecture retenue. Les contrôles ont porté sur la cohérence observée après mutations : création d'une affectation suivie d'un GET détail projet montrant le talent listé; création d'une demande RH suivie de son apparition dans GET

/webhook/manager/rh-actions et traitement RH ; enregistrement d'une décision Copilot visible dans GET /webhook/manager/copilot-decisions.

6.5.2 Contraintes relationnelles

Les contraintes d'identifiants UUID sont renforcées côté client par `assertUuid` (`manager-api-contra` `rh-actions.api.ts`) et par la sanitization what-if dans `orchestrator.api.ts`. Des envois avec identifiant mal formé ont été utilisés pour vérifier que l'UI bloque ou que le serveur renvoie une erreur explicite (`user-facing-api-error.ts`) plutôt qu'un état incohérent silencieux.

6.5.3 Cohérence des calculs

La cohérence des calculs de viabilité, de risque et de matching est validée par comparaison entre la valeur affichée dans l'UI et la réponse brute de POST /webhook/api/project/viability ou du détail projet, sans modification intermédiaire dans React. Le principe `crud-domain.ts` a été vérifié par inspection de code et par absence de fonctions de recalcul dans les hooks de portefeuille : les scores affichés proviennent des champs serveur `viability_score`, `risk_score`, `decision`.

Les seuils métier documentés (Continue si score $\geq 7,5$, Adjust entre 4 et 7,5, Stop en dessous de 4) sont appliqués côté n8n ; la recette consiste à constater que le libellé renvoyé dans `decision` est l'un des trois et correspond à l'ordre de grandeur du score affiché.

6.6 Cas d'usage complet

6.6.1 Analyse d'un projet réel

Le cas d'usage retenu pour la démonstration de bout en bout est le pilotage d'un projet existant sur l'environnement de préproduction : ouverture depuis `ProjectsPageManager`, chargement du modal Mission Control, consultation des onglets overview et équipe, déclenchement d'un recompute ou d'une analyse détaillée si les données sont périmées (traces stale ou aging).

6.6.2 Calcul du score de viabilité

L'action POST /webhook/api/project/viability avec `use_ai: true` produit un score sur dix et une décision. L'UI met à jour les cartes de viabilité et l'onglet décisions sans

recalcul local. La validation consiste à comparer le score JSON et l’affichage, et à vérifier la présence d’une explication textuelle lorsque le workflow la fournit.

6.6.3 Recommandation Continue / Adjust / Stop

Les trois libellés `DecisionLabel` sont affichés dans le journal (`DecisionLogPage`, graphiques `Recharts`) et dans `Mission Control`. La recette a couvert l’enregistrement d’une décision utilisateur via `saveCopilotDecision` ou `postStrategicDecision` et, le cas échéant, l’exécution d’une option `Strategist` (`strategist/execute`) avec retour métier (`WF_Strategist_Accept` pour `Stop` ou réduction de périmètre).

6.6.4 Simulation What-if

Le scénario what-if ouvre `WhatIfProvider`, saisit une modification d’allocation ou de délai, appelle `POST /webhook/api/project/what-if` et affiche `WhatIfResultPanel` avec `score_before`, `score_after`, `delta`, `decision_before`, `decision_after` et recommandations structurées (`WhatIfResponse` dans `api.types.ts`). La figure 6.1 rappelle la structure cible du contrat.

```
status: success | score_before / score_after / delta
decision_before / decision_after | score_breakdown_*
recommendation.summary | risks[] | scenario_summary
```

FIGURE 6.1 – Champs principaux attendus d’une réponse what-if (`WhatIfResponse`)

La validation manuelle vérifie que le panneau tolère les champs partiels mentionnés dans le commentaire du type TypeScript lorsque le workflow renvoie une réponse incomplète.

6.7 Résultats obtenus

6.7.1 Résultats fonctionnels

Sur le plan fonctionnel, les parcours prioritaires du cahier des charges ont été rendus opérationnels sur l’environnement de préproduction : trois workspaces, `Mission Control`, alertes et demandes RH, `Copilot` et `Helper`, simulateur what-if, rapports PDF et journal des décisions. Les écarts restants concernent surtout la dépendance à la qualité des données en base et à la disponibilité des workflows n8n, plutôt que des blocages structurels de l’interface `React`.

6.7.2 Résultats techniques

Sur le plan technique, `npm run typecheck` et `npm run build:check` s'exécutent sans erreur de typage sur le périmètre du dépôt au moment de la rédaction. Le proxy Vite et le relais Vercel permettent d'exercer les webhooks sans CORS bloquant. Le test unitaire `agent-trace.test.ts` valide la classification temporelle des traces. En revanche, la couverture de tests automatisés reste faible : un seul fichier de test pour une base de code de plusieurs centaines de composants et hooks.

6.7.3 Apports métier

Les retours des encadrants et les démonstrations de stage ont mis en évidence un apport principal : la consolidation, pour le manager, des signaux projet, risques et staffing dans Mission Control, avec une décision suggérée explicite (Continue, Adjust, Stop) plutôt qu'une dispersion entre feuilles de calcul. Le flux de demandes RH structuré améliore la traçabilité entre ligne managériale et fonction RH. L'espace talent offre une visibilité lecture seule cohérente sur les missions et la charge, sans exposer les fonctions de pilotage stratégique.

6.8 Limites observées

Plusieurs limites ont été observées et sont assumées dans le périmètre du PFE. L'absence de suite de tests end-to-end automatisés rend les régressions dépendantes de recettes manuelles. L'environnement n8n partagé et les données de préproduction ne permettent pas toujours de reproduire de manière déterministe un même score de viabilité entre deux exécutions, notamment lorsque `use_ai: true` active un modèle génératif. La latence des workflows peut dépasser une minute sur certaines analyses, ce qui exige des timeouts élevés et peut frustrer l'utilisateur malgré les indicateurs de chargement React Query.

Les réponses n8n parfois encapsulées ou partiellement conformes au type TypeScript ont nécessité des parseurs défensifs ; un écart de contrat backend-front peut encore produire un affichage vide ou un message générique. Enfin, la validation PostgreSQL directe (contraintes FK, index) n'a pas fait l'objet d'un protocole formalisé dans ce dépôt front-end.

6.9 Perspectives d'amélioration

Les perspectives d'amélioration les plus pertinentes découlent directement des limites constatées. L'introduction d'une suite de tests end-to-end (Playwright) sur les parcours login, Mission Control et demande RH sécuriserait les livraisons. L'archivage d'une collection

Postman ou d'un fichier OpenAPI généré depuis n8n faciliterait la régression API. L'extension des tests unitaires aux parseurs (`flattenN8nRow`, `rh-api-parse.ts`) et aux validateurs UUID réduirait les risques de régression silencieuse.

Côté produit, un environnement de démo dédié avec jeu de données figé améliorerait la reproductibilité des scores. Des indicateurs de progression plus granulaires pendant les workflows longs et des tests de charge sur les webhooks critiques renforceraient la robustesse SaaS. L'harmonisation progressive des chemins versionnés et UUID vers une convention unique simplifierait la configuration `VITE_*`.

6.10 Conclusion

Ce chapitre a présenté la stratégie de test de Strategic Copilot, fondée sur la recette manuelle des trois workspaces, la validation des webhooks n8n sur l'instance de préproduction, les contrôles statiques TypeScript et le test unitaire des traces d'agents. Les tableaux ont explicité la nature des preuves afin de ne pas surévaluer l'automatisation réellement présente dans le dépôt. Les cas d'usage viabilité, décision Continue/Adjust/Stop et what-if ont été décrits au regard des contrats `api.types.ts` et des écrans livrés.

Globalement, le système atteint les objectifs fonctionnels du stage sur l'environnement testé, avec une marge de progrès importante sur l'automatisation des tests et la reproductibilité des analyses IA. La conclusion générale du rapport reviendra sur la valeur du livrable Augmented Talents et sur les suites de travail recommandées avant une mise en production élargie.

Conclusion générale

Ce projet de fin d'études s'est inscrit dans un contexte où la transformation numérique pousse les organisations à consolider des données RH et projet de plus en plus volumineuses, tout en exigeant des décisions plus rapides, plus traçables et mieux partagées entre les acteurs. Au sein de l'écosystème **Augmented Talents**, le travail réalisé a porté sur **Strategic Copilot** : une plateforme SaaS d'aide à la décision pour la gestion stratégique des talents et du portefeuille projet. L'ambition était de rapprocher, dans une interface unique, la visualisation des indicateurs, la détection des risques, l'analyse de viabilité des projets, les recommandations assistées par intelligence artificielle et la gestion structurée des demandes RH, pour les profils manager, responsable RH et talent. Encadré pédagogiquement par Madame Soundouss Abroun à l'École Marocaine des Sciences de l'Ingénieur de Tanger, et professionnellement par Monsieur Youssef Moufak ainsi que par Messieurs Hamza Sadouk, Hicham Taoufik et Mohyi Eddine Saadani, le stage a visé à livrer une solution opérationnelle, fondée sur le dépôt *frontend-ui* et sur le document de cadrage *PFE Agentic AI Strategic Talents*.

Le rapport a retracé cette démarche en six chapitres complémentaires. Après le contexte et l'analyse des besoins, l'étude bibliographique a posé les fondements RH, projet, aide à la décision et technologies web. Le cahier des charges et la planification ont formalisé les spécifications et le calendrier du PFE. La conception a décrit l'architecture multi-agents, les workflows n8n et les API. La réalisation a détaillé l'implémentation React/Vite et les interfaces livrées. Enfin, le chapitre consacré aux tests a explicité la stratégie de validation et les limites de la preuve automatisée. Cette conclusion générale en tire le bilan global.

Sur le plan des **résultats**, les objectifs fixés en début de projet ont été atteints dans une large mesure sur l'environnement de préproduction connecté à `n8nprod.aphelionxinnovations.com`. L'application *copilot-ui* propose trois workspaces cohérents (`/workspace/manager`, `/workspace/rh`, `/workspace/talent`), protégés par authentification JWT et contrôle d'accès par rôle. Le manager dispose d'un tableau de bord, d'un portefeuille projets, du modal *Mission Control* (pilotage à 360 degrés : overview, équipe, tâches, risques, simulation, décisions), de la gestion des alertes, des demandes RH, du Helper conversationnel, du simulateur what-if et du journal des décisions enrichi de graphiques Recharts. Le responsable RH accède aux agrégats métier, au référentiel talents et compétences, aux plans de

formation, à la mobilité, aux alertes organisationnelles et à la gestion des comptes. Le talent consulte ses missions, tâches, charge, compétences et formations via des endpoints dédiés `/api/workspace/talent/*`.

L'architecture retenue respecte le principe **API-first** et la règle, inscrite dans `crud-domain.ts`, selon laquelle le front-end n'effectue aucun recalcul de scores de viabilité, de risque ou de recommandations : ces traitements sont délégués aux workflows n8n, qui orchestrent les agents documentés (analyse projet, matching talents, risques et signaux faibles, orchestrateur de viabilité, Helper Copilot) et persistent les résultats dans PostgreSQL, hébergé via Supabase. Les scores et décisions **Continue**, **Adjust** et **Stop** sont produits côté serveur ; l'utilisateur conserve la responsabilité de la validation, notamment via le Strategist, les demandes RH et l'enregistrement des décisions Copilot. Les campagnes de validation menées pendant le stage — recettes manuelles des parcours critiques, contrôles `npm run typecheck` et `npm run build:check`, test unitaire des traces d'agents — confirment la cohérence fonctionnelle de cette chaîne sur les données et workflows disponibles en préproduction.

Ce travail a permis de consolider des **compétences techniques** variées. Sur le plan front-end, la maîtrise de React 19, TypeScript, Vite, Tailwind CSS 4, React Router 7 et TanStack React Query s'est traduite par une SPA multi-rôles maintenable, avec internationalisation (français, anglais, arabe) et accessibilité via React Aria. L'intégration de dizaines de webhooks versionnés (`wmp-*v1`, `wmt-*v1`, routes `/webhook/api/*`) a exigé une rigueur sur les contrats JSON, les parseurs défensifs et la configuration par variables `VITE_*`. La mise en place du proxy de développement et du relais Vercel a renforcé la compréhension des enjeux CORS, de sécurité JWT et de déploiement SaaS.

Sur le plan **méthodologique**, ce stage m'a permis de mener un projet complet, de l'analyse des besoins à la validation : cadrage agentique (Analyst, Matchmaker, Watchdog, Orchestrator), conception de l'architecture, développement frontend et backend, modélisation de la base de données, intégration de l'IA et recettes sur l'environnement de préproduction. La rédaction du présent rapport en LaTeX modulaire a structuré la communication écrite de ce travail. Les échanges réguliers avec les encadrants ont contribué à aligner les choix techniques sur des objectifs métier explicites.

Plusieurs **limites** demeurent toutefois assumées. La couverture de tests automatisés reste faible dans le dépôt (essentiellement `agent-trace.test.ts`), ce qui rend les régressions dépendantes de recettes manuelles. L'environnement n8n et les données de préproduction partagés limitent la reproductibilité stricte des scores lorsque l'IA générative est activée (`use_ai: true`). La latence de certains workflows impose des timeouts élevés et peut dégrader l'expérience utilisateur. La multiplicité des conventions d'URL (chemins versionnés, webhooks UUID) complexifie la configuration et la maintenance. Enfin, la qualité des recommandations et des alertes reste conditionnée par l'exhaustivité et la fraîcheur des données en base, facteur hors du seul front-end mais déterminant pour l'adoption terrain.

Les **perspectives** d'évolution s'articulent naturellement autour de ces limites. À court terme, l'extension des tests automatisés (Playwright, tests unitaires sur les parseurs API) et l'archivage d'une spécification OpenAPI ou d'une collection Postman sécuriseraient les livraisons. À moyen terme, un environnement de démonstration avec jeu de données figé, des indicateurs de progression plus fins pendant les analyses longues et une harmonisation des chemins n8n simplifieraient l'exploitation. Sur le plan produit Augmented Talents, l'enrichissement des scénarios what-if, le renforcement de la gouvernance des décisions IA et l'élargissement des rapports PDF automatisés pourraient accroître la valeur perçue par les directions RH et opérationnelles. L'intégration continue entre workflows n8n et interface React demeure le fil conducteur : faire évoluer les agents sans remettre en cause l'expérience utilisateur, grâce à des contrats stables et à une traçabilité exemplaire.

En définitive, Strategic Copilot démontre la faisabilité d'un copilote stratégique agentique au service du pilotage conjoint des talents et des projets. Le livrable **copilot-ui** matérialise une architecture moderne, découplée et orientée aide à la décision responsable, où l'humain reste au centre des arbitrages. Pour l'auteur de ce mémoire, Ouissame Mahtour, ce projet de fin d'études constitue une synthèse exigeante entre ingénierie logicielle, gestion des ressources humaines et intelligence artificielle appliquée ; il ouvre sur une contribution durable à la plateforme Augmented Talents et sur une pratique professionnelle renforcée par la rigueur, la curiosité technique et le souci de la valeur métier.

Bibliographie

- [1] David Arnott and Graham Pervan. A critical analysis of decision support systems research. *Journal of Information Technology*, 20(2) :67–87, 2005.
- [2] Gary S. Becker. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. University of Chicago Press, Chicago, 3 edition, 2009. Fondements économiques du capital humain.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33 :1877–1901, 2020. Présentation du modèle GPT-3.
- [4] Robert G. Cooper, Scott J. Edgett, and Elko J. Kleinschmidt. *Portfolio Management for New Products*. Basic Books, New York, 2 edition, 2016.
- [5] Janet H. Marler and John W. Boudreau. An evidence-based review of HR analytics. *The International Journal of Human Resource Management*, 24(1) :14–33, 2013. Analytique RH et aide à la décision en ressources humaines.
- [6] Meta Platforms, Inc. React – a javascript library for building user interfaces. <https://react.dev/>, 2024. Documentation officielle. Consulté en 2025–2026.
- [7] Microsoft Corporation. Typescript – javascript with syntax for types. <https://www.typescriptlang.org/>, 2025. Documentation officielle. Consulté en 2025–2026.
- [8] n8n GmbH. n8n – workflow automation for technical teams. <https://docs.n8n.io/>, 2025. Orchestration de workflows et webhooks. Consulté en 2025–2026.
- [9] OpenAI. Openai platform documentation – api reference. <https://platform.openai.com/docs>, 2025. Modèles de langage et API utilisés côté workflows n8n. Consulté en 2025–2026.
- [10] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Project Management Institute, Newtown Square, PA, 7 edition, 2021. Référentiel Project Portfolio Management et pilotage projet.
- [11] Recharts Group. Recharts – a composable charting library built on react components. <https://recharts.org/>, 2025. Graphiques du journal des décisions. Consulté en 2025–2026.

- [12] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, Hoboken, NJ, 4 edition, 2021.
- [13] Rob Silzer and Ben E. Dowell. *Strategy-Driven Talent Management : A Leadership Imperative*. Jossey-Bass, San Francisco, 2010.
- [14] Herbert A. Simon. *Administrative Behavior : A Study of Decision-Making Processes in Administrative Organizations*. Free Press, New York, 4 edition, 1997. Rationalité limitée et processus décisionnel.
- [15] Michael Stonebraker and Robert Cattell. Ten rules for scalable performance in simple operation. *Communications of the ACM*, 54(4) :56–64, 2011.
- [16] Stefan Strohmeier. *Digital Human Resource Management : A Research Agenda*. Springer, Cham, 2020. Transformation digitale des systèmes d’information RH.
- [17] Supabase Inc. Supabase – the postgres development platform. <https://supabase.com/docs>, 2025. Hébergement PostgreSQL, authentification et stockage objet. Consulté en 2025–2026.
- [18] Tailwind Labs Inc. Tailwind css – rapidly build modern websites. <https://tailwindcss.com/>, 2025. Documentation officielle. Consulté en 2025–2026.
- [19] The PostgreSQL Global Development Group. Postgresql 16 documentation. <https://www.postgresql.org/docs/>, 2025. Système de gestion de bases de données relationnelles. Consulté en 2025–2026.
- [20] Edward R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, Cheshire, CT, 2 edition, 2001. Principes de visualisation pour tableaux de bord décisionnels.
- [21] Efraim Turban, Ramesh Sharda, and Dursun Delen. *Decision Support and Business Intelligence Systems*. Pearson, Boston, 10 edition, 2018.
- [22] Evan You and Vite Contributors. Vite – next generation front-end tooling. <https://vite.dev/>, 2025. Documentation officielle. Consulté en 2025–2026.

Annexes

Les annexes regroupent les éléments complémentaires du projet *Strategic Copilot* qui, par leur nature technique ou illustrative, ne peuvent être exposés en détail dans le corps du rapport sans alourdir la lecture. Elles visent à documenter de manière reproductible les artefacts produits ou mobilisés au cours du stage : requêtes et structures de données, contrats d'échange entre le front-end React et les services orchestrés par n8n, interfaces utilisateur livrées dans le dépôt `copilot-ui`, ainsi que les schémas d'architecture et de flux complémentaires.

Le lecteur y trouvera notamment :

- des **extraits de code SQL** illustrant le modèle relationnel PostgreSQL (tables talents, compétences, projets, scores de viabilité, journal des décisions et alertes) ;
- des **exemples de réponses JSON** renvoyées par les webhooks n8n consommés par l'application (`/webhook/manager/...`, `/webhook/api/project/...`) ;
- des **captures d'écran des interfaces** par espace métier (manager, RH, talent) : tableaux de bord, portefeuille projet, fiche talent, notifications, simulateur what-if et assistant Copilot ;
- des **extraits de workflows n8n** décrivant les processus P0 à P5 (synchronisation des données, analyse projet, matching, risques, orchestration stratégique, simulation) ;
- des **diagrammes complémentaires** (architecture logicielle, enchaînement des agents, séquences d'appels API) venant compléter ceux présentés dans les chapitres de conception et de réalisation.

Ces documents permettent de vérifier la cohérence entre les spécifications du cahier des charges, l'implémentation effective et les démonstrations réalisées lors des phases de tests. Ils sont numérotés ci-dessous pour faciliter les renvois depuis le corps du rapport.

Annexe A — Extraits de code SQL

Cette annexe présente des requêtes et définitions de schéma PostgreSQL utiles à la compréhension du modèle de données (référentiels RH, affectations projet, persistance des

scores analytiques et des alertes).

Annexe B — Exemples de réponses JSON

Cette annexe regroupe des exemples de payloads échangés entre `copilot-ui` et les endpoints `n8n` (tableau de bord manager, détail projet, alertes de risque, actions RH, résultats what-if).

Annexe C — Captures d'écran des interfaces

Cette annexe illustre les principaux écrans de la plateforme : espaces `/workspace/manager`, `/workspace/rh` et `/workspace/talent`, ainsi que les composants transverses (notifications, journal des décisions, Copilot intégré).

Annexe D — Extraits de workflows n8n

Cette annexe documente des extraits des workflows d'orchestration (`WF_Data_Sync`, `WF_Project_Analysis`, `WF_Talent_Matching`, `WF_Risk_KPI`, `WF_Strategic_Orchestrator`, `WF_What_If`) et leurs points d'entrée webhook.

Annexe E — Diagrammes complémentaires

Cette annexe complète les figures du rapport par des vues d'architecture (couches présentation, orchestration, persistance), de déploiement et de séquence entre acteurs (utilisateur, front-end, `n8n`, Supabase, modèle IA).